UNIVERSITY OF MILANO-BICOCCA

Department of Computer Science, Systems and Communication

Master's Degree Programme in **Computer Science**

Innovative Management of Service Requests: From a Distributed Monolith to a Large-Scale Microservices Architecture, with RAG-Enhanced Reccomendation System

Advisor: Prof. Leonardo Mariani

Master's Thesis by:
Nicolas Guarini
Student ID: 918670

Academic Year 2024–2025

"What I cannot create, I do not understand"

- Richard Feynman

Contents

Introduction

The technological evolution of recent decades has profoundly transformed the business landscape, making IT solutions a crucial factor for the success of enterprises. In this context, Elmec Informatica S.p.A. stands out as a significant player in the Information Technology sector in Italy. Founded in 1971 and headquartered in Varese, Elmec combines extensive industry experience with a strong focus on innovation, offering services and solutions that cover a wide range of business needs, offering advanced IT solutions for medium and large enterprises.

The company manages four datacenters (including one Tier IV certified), provides Hybrid IT services, digital workspace and Device-as-a-Service solutions, and is a key partner for cybersecurity and regulatory compliance. With over 450 certified technicians, Elmec integrates cloud technologies (in collaboration with AWS and GAIA-X), promotes environmental sustainability, and stands out for its innovation through its Technological Campus and ongoing investments in professional training.

The primary context of this internship revolves around the redesign and enhancement of the existing *Service Request Portal (SRP)*, which is essential for managing client service requests efficiently.

A Service Requests is a formal request submitted by a customer to obtain specific services or actions within their IT infrastructure. These requests can encompass a wide range of activities, from creating new user accounts in the Active Directory system to managing user permissions, resetting passwords, and deploying software updates.

Various challenges have been identified within the current architecture, particularly concerning the dynamic forms used by customers for submitting the service requests, the management of customizable fields, and the integration with external data sources.

The main objectives of this internship are:

- **Analysis of the Existing System**: Conduct a comprehensive analysis of the current state of the SRP system, identifying weaknesses and inefficiencies that hinder its performance and user experience;
- **Architectural redesign**: Propose and implement a complete architectural redesign of the system;

- **Modular and Scalable Solution**: Design a modular and scalable solution that can accomodate various client-specific configurations;
- **Migrations**: Plan and execute a full migration of databases and other company services/platforms that use SRP.
- **Fine-tuned LLMs integration**: Explore opportunities for integrating specifically fine-tuned LLM systems into the SR workflow.

Chapter 1

Overview of the existing System

The internship project was not focused on creating an entirely new system from scratch, but rather on a deep redesign and restructuring of an existing system developed many years ago using legacy technologies and affected by several issues and critical limitations.

Before discussing the architectural, technological, and implementation choices that were made, it is necessary to carry out a thorough analysis of the current system, in order to understand its behavior, operational flows, technologies used, the main issues to be addressed, how to resolve them, how to prevent them from reoccurring in the future, and how to ensure that the new system is scalable and maintainable.

1.1 Service Requests Workflow

The lifecycle of a Service Request within the current system is orchestrated through a series of defined states and automated processes, involving interactions between various components and human operators. The flow is designed to manage SRs from initiation to completion, handling approvals, execution, and potential errors [27].

In Figure 1.1 [27], a simple and high-level flowchart is shown that illustrates the main flows and scenarios.

1.1.1 SR Initiation and Initial State Management

A Service Request can be initiated through two primary channels [27]:

- **Via MyElmec Portal**: Clients can submit an SR by completing a dedicated form from the MyElmec portal. Upon saving the form, a ticket is created in the HelpdeskAdvanced (HDA) system with a `QUALIFIED` status, and a corresponding Service Request is generated in SRP with a `NEW` status. These two entities are then linked via the HDA Ticket ID;
- **Via HDA Ticket by Elmec Operator**: Operators have the ability to associate a catalog SR directly with a ticket they are managing within the HDA interface. When the ticket is saved in a `DELEGATED` status, the SR

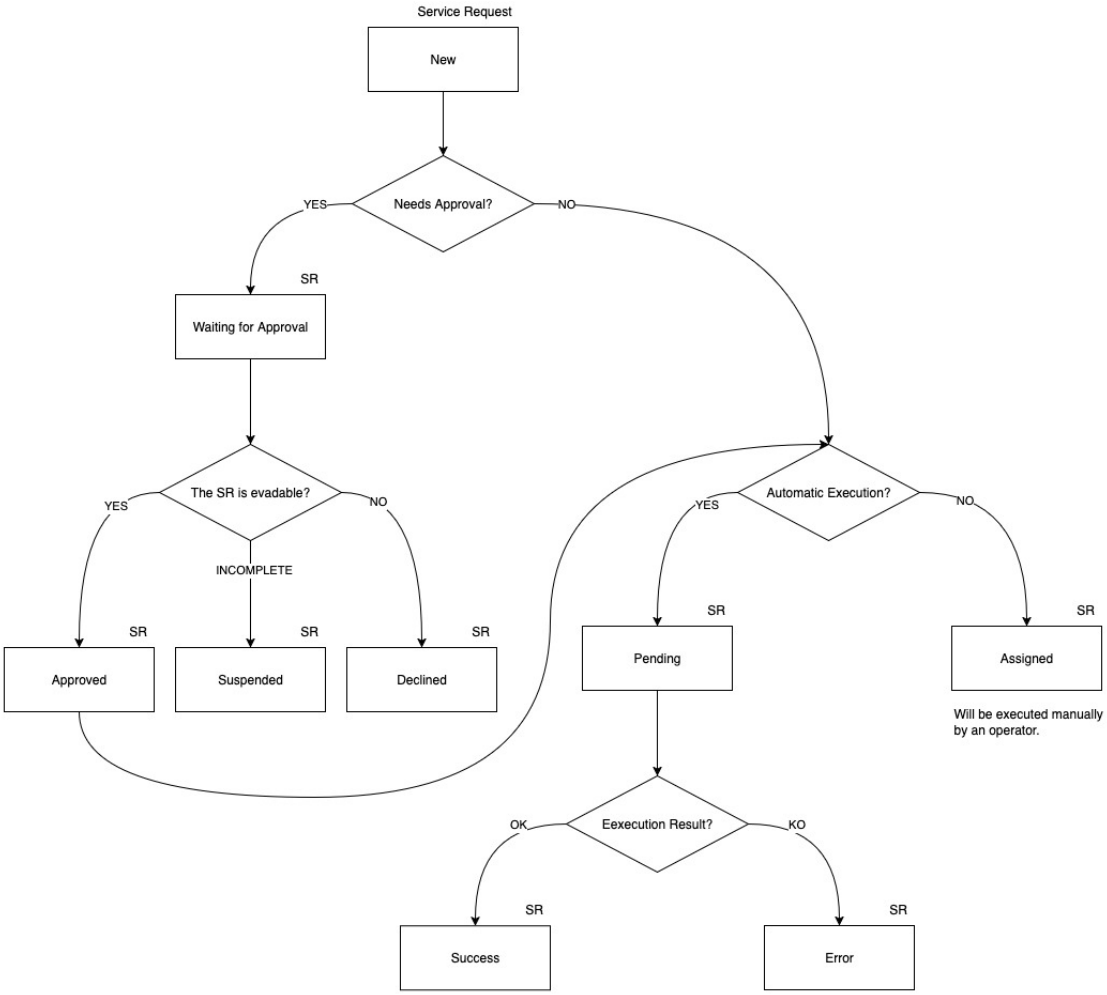

Figure 1.1: High-Level Service Request Lifecycle

is created in SRP with a `NEW` status, linked to the ticket, and immediately triggers the SR workflow within SRP.l

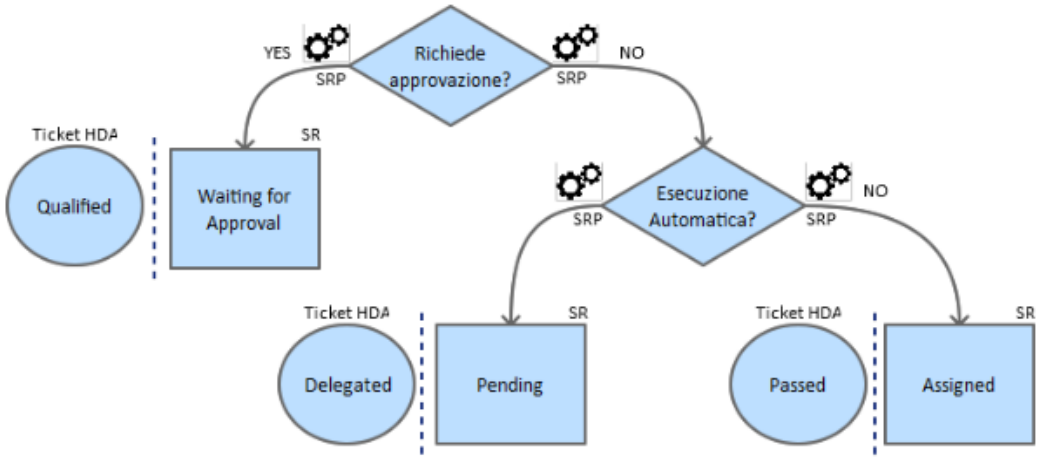

Figure 1.2: AS-IS New Service Request Workflow

As shown in Figure 1.2, following the Service Request creation, the system evaluates whether the Service Request requires approval by the operators. The logic for setting the initial SR and HDA ticket states is as follows [27]:

- **If approval is required**: the Service Request is set to `WAITING FOR APPROVAL` status, and the corresponding HDA ticket is set to `QUALIFIED`. A communication is added to the ticket, indicating that the SR needs validation from an operator before execution;
- **If no approval is required and it's linked to an automatism**: the Service Request enters a `PENDING` state, and the HDA ticket is set to `DELEGATED`;
- **If no approval is required and it's not linked to an automatism**: the Service Request is `ASSIGNED`, and the HDA ticket is set to `PASSED`. A communication is added to the ticket, noting that the SR requires manual processing by an operator.

1.1.2 Approval and Execution Workflows

As shown in Figure 1.3, for SRs requiring approval (`WAITING FOR APPROVAL` state with an HDA `QUALIFIED` ticket), an operator's intervention is necessary to validate the request. The operator has three possible actions [27]:

- **Cancel the SR**: the HDA ticket is set to `CLOSED ABORTED`, the SR status becomes `DECLINED`, and a notification email is sent to the requesting client;
- **Request client modification**: the HDA ticket is set to `WAITING USER`, the SR status becomes `SUSPENDED`, and a notification email is sent to

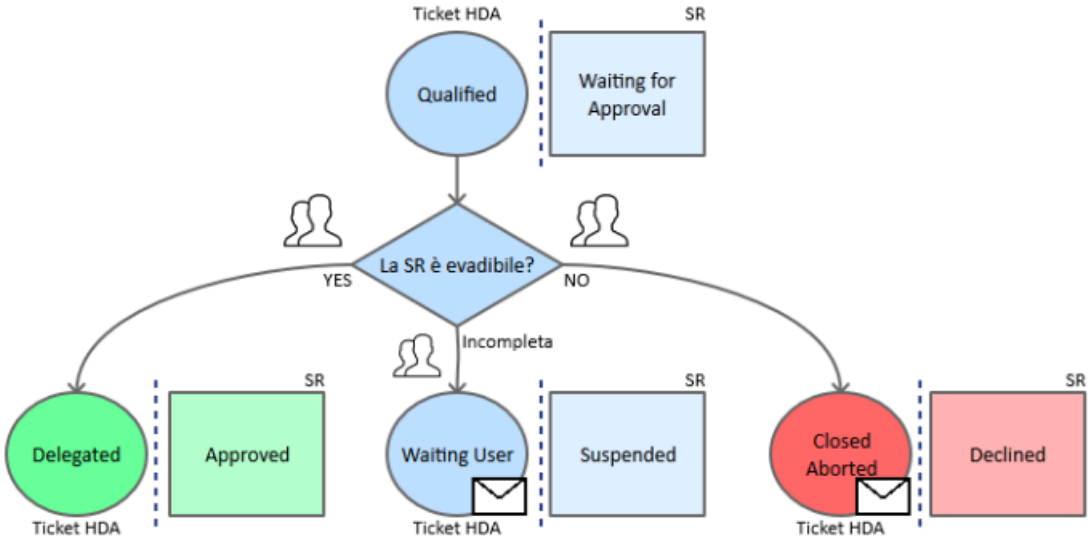

Figure 1.3: AS-IS Approval Service Request Workflow

the client. The workflow pauses until the client modifies the SR via Elmec.com;

- **Approve the SR**: the HDA ticket is set to `DELEGATED`, and the SR status becomes `APPROVED`.

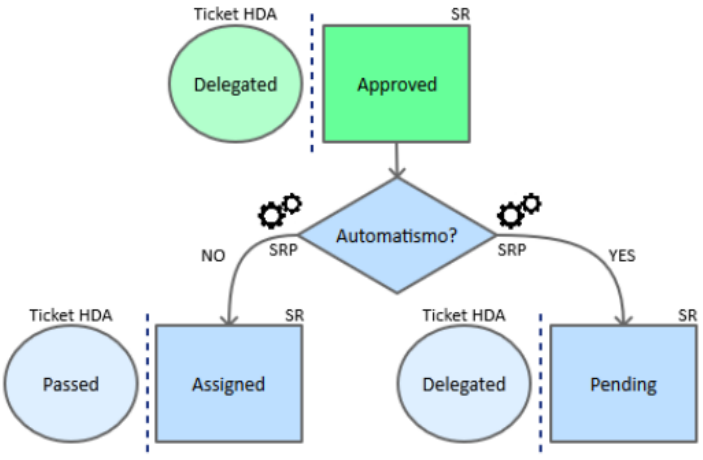

Figure 1.4: AS-IS Approved Service Request Workflow

As shown in Figure 1.4, once a SR is approved, SRP manages its subsequent flow based on whether it is linked to an automatism [27]:

- **If the SR is linked to an automatism**: the SR transitions to a `PENDING` state, while the HDA ticket remains `DELEGATED`. This implies it's awaiting automated execution;

- **If the SR is not linked to an automatism**: the SR is immediately set to `ASSIGNED`, and the HDA ticket is moved to `PASSED`. A note is added

to the ticket indicating that the SR requires manual processing.

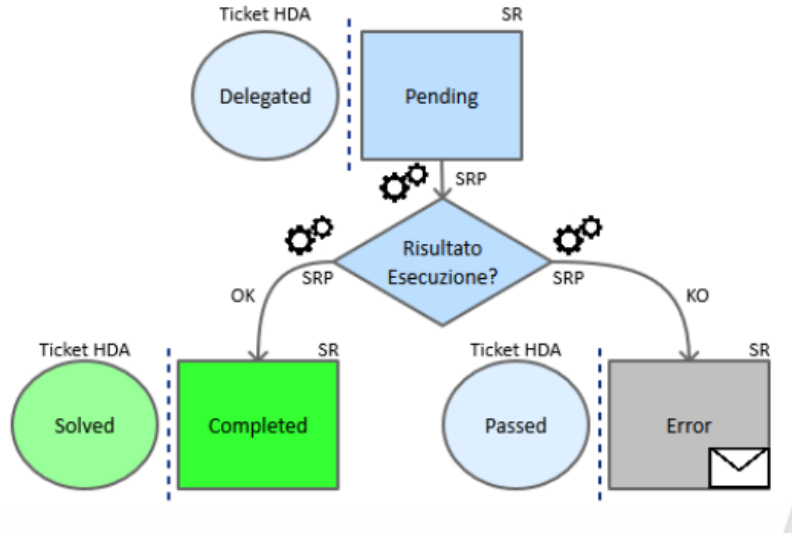

Figure 1.5: AS-IS Pending Service Request Workflow

As shown in Figure 1.5, an SR in `PENDING` state is awaiting the execution of its associated automatism. The outcome of this automated execution determines the next state [35]:

- **Successful execution**: the SR is set to `COMPLETED`, and the corresponding HDA ticket is set to `SOLVED`;
- **Unsuccessful execution**: the SR transitions to an `ERROR` state, and the HDA ticket is set to `PASSED`. Communication regarding the error is added to the ticket for visibility within HDA.

The execution of Service Requests themselves can involve the SRP Agent on the client's Domain Controller for operations requiring client-side interaction (e.g., Active Directory or Exchange environment changes). The SRP Agent periodically checks `ELMEC-SRP01` for pending SRs. If found, it retrieves the SR details, loads the relevant PowerShell script, and executes it. The agent then informs ELMEC-SRP01 that the SR is running [35]. Monitoring is performed by a scheduled `READLOG` task on the client side, which checks the PowerShell script's log file every minute. A "KEY" in the log indicates completion, prompting the SRP Agent to inform ELMEC-SRP01 to set the SR to COMPLETED, triggering an HDA status update. An "ERROR" key indicates failure, leading to the log being sent to ELMEC-SRP01 and an error signal. A timeout mechanism also signals an error if completion is not detected within a specified duration [35].

A constraint that must be respected is that the high-level flow (state transitions and mapping between SR states and HDA ticket states) must not be modified, as it represents an established and approved process that also encompasses organizational and accountability aspects among various Elmec departments

[28]: those who configure and manage the catalog and related Service Requests (Managed Services (MxS) team), those who develop and maintain the product (Innovation team), and those responsible for the ticket management system (HDA team). The technological and implementation aspects, on the other hand, can and should be redesigned and improved [28], as will be explored in the following chapters.

1.2 Current System Architecture

The *SRP* system is fundamentally divided into two primary segments:

- Company Network;
- Client Network.

These segments interact and communicate exclusively through a *DMI Console* [29], which acts as a unidirectional bridge, transferring requests from the client network to the internal company network [29] [13].

The overall architecture is depicted in the diagram in figure 1.6 [29].

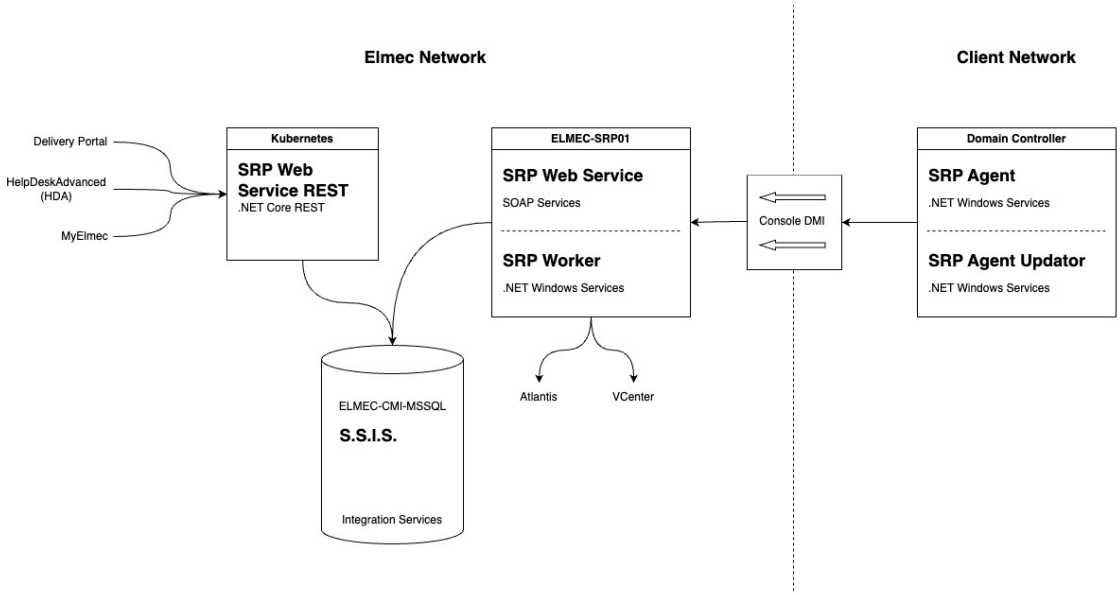

Figure 1.6: AS-IS System Architecture

The system's operational flow involves several key components across both networks, orchestrating the processing of service requests.

1.2.1 Main Functionalities and Components

The current *SRP* system relies on a suite of interconnected components, each serving a specific role in the lifecycle of a service request.

Elmec's Network Components:

- **SRP Web Sevice REST** [34]: This component, deployed on a Kubernetes cluster, exposes .NET Core RESTful services [29]. It serves as the primary interface for external systems such as:
  - **Delivery Portal**: An asset management system used to manage Service Request catalog configuration and Elmec's internal and client servers;
  - **HDA (HelpDeskAdvanced)**: A ticket management system. Each service request is intrinsically linked to an HDA ticket via an ID, and HDA is responsible for initiating and associating service requests with existing tickets.
  - **MyElmec**: A client-facing portal enabling customers to create, manage, and monitor the various services provided by Elmec (e.g., servers, management software, cloud solutions).

  The *SRP Web Service REST* communicates with the *ELMEC-CMI-MSSQL* server.
- **ELMEC-CMI-MSSQL**: This server hosts the central SQL Server Database for the SRP system. It is the persistent storage layer for all service request data and related information [29].
- **S.S.I.S. (SQL Server Integration Services)**: Running on the ELMEC-CMI-MSSQL server, SSIS packages are crucial for data transformation and import/export operations. They facilitate the transfer of data, such as master data (e.g., Active Directory users, Organizational Units, groups) sent by SRP Agents, into the database. Each SSIS package is designed to transform the received information for database insertion or vice versa [29].
- **ELMEC-SRP01**: This Windows Server functions as the core engine for service requests within the Elmec network. It houses all the PowerShell scripts associated with individual service requests, which are maintained by the Platform team [29]. Key components residing on ELMEC-SRP01 include:
  - **SRP WebServiceSoap**: This component provides SOAP services primarily for communication with the SRP Agent located on the client side. It interacts with the database to retrieve necessary information before exposing it to the agents for service request execution [33].
  - **SRP InternalWorker**: A Windows service that operates similarly to the SRP Agent but executes entirely within the Elmec network. This is utilized for service requests that do not require interaction with the client's network or Active Directory (e.g., "no patch" service

requests). The InternalWorker queries the database every 10 seconds for "internal worker" type jobs (like removing a server from a patching session, or resetting a managed Virtual Machine), connects to the SOAP service, retrieves the Service Request ID and its fields, and initiates job execution. It also monitors logs every 30 seconds to determine the job's status [32].

The only component in the client's network is called **Domain Controller**, which represents the client's Active Directory domain controller, where the communication tools with Elmec are installed. It hosts:

- **SRP Agent (.NET Windows Service)**: Typically, one agent is configured per client domain, specifically for Active Directory (AD) and Exchange-related service requests. This agent performs two main functions [35]:
  - Every 8 hours, it reads master data (AD users, OUs, groups) from the domain controller and sends it to ELMEC-SRP01 for storage via SSIS.
  - Every 5 minutes, it queries ELMEC-SRP01 for pending Service Requests. Upon finding any, it requests detailed information, loads the corresponding PowerShell script, and executes it. After initiating the script, the agent informs ELMEC-SRP01 that the SR is "running" and proceeds to the next SR.
- **SRP Agent Updator (.NET Windows Service)**: This service continuously monitors the SRP Agent folder for a file with a .new extension. If detected, it stops the SRP Agent, updates it, and restarts it

1.2.2 Interactions and Communication between Customers and Company networks

The Communication between the company network and the clients' networks is a critical aspect of the SRP system's operations and is handled through the **DMI Console**.

The DMI console serves as a key infrastructural component, primarily acting as a communication bridge between the customer's network and Elmec's internal network. Specifically, for the SRP system, the DMI console is responsible for transferring service requests (SR) from the customer's network to Elmec's internal network, although it does not handle traffic in the opposite direction [13].

Operation of the DMI Console

The connectivity of the DMI appliances to Elmec is established through multiple OpenVPN connections, initiated by the appliance towards the Brunello 4 and Mendrisio Data Centers (for Swiss customers). The DMI consoles are not directly exposed to the Internet and are accessible only by authorized personnel. Hardware requirements for a DMI node include 4 GB of RAM, 4 cores, and 50 GB of disk space [13].

Following recent developments in the DMI, the new consoles use K3S for container management. K3S is a lightweight, certified Kubernetes distribution designed for edge, IoT, and CI/CD environments. It is characterized by being a single binary that reduces external dependencies and uses SQLite as the default database, greatly simplifying installation and management. It already includes essential components such as *containerd* [17].

All consoles are centrally managed via Rancher, and the deployment of the application stack is delegated to ArgoCD, which ensures that container versions are constantly updated. For security reasons, the console in the customer network by default exposes only SSH, while services such as SRP are selectively enabled and only when necessary. All application data present on the consoles is stored on a LUKS (Linux Unified Key Setup) encrypted volume[1], which can only be unlocked if the VPN tunnel is established. The operating system update is performed through a standard Elmec system, called PMD (Patching Management Dashboard) [13].

1.3 Problems and core issues of the current system

The analysis of the architecture and workflow of the current SRP system has revealed a number of critical issues that compromise its scalability, maintainability, and reliability. These problems are deeply rooted in the technological and design choices made years ago and are evident across several key areas of the system, from catalog management to script execution and the import of master data from Active Directory.

[1]A platform-independent hard disk encryption method that can be used to encrypt disks across a wide variety of tools. This not only ensures full compatibility and interoperability between different software but also guarantees that password management occurs in a secure and documented manner [9].

1.3.1 Architectural and Technological Challenges

The SRP system is characterized by a fragmented architectural structure and complex interdependencies between its main components, which leads to several critical issues.

Monolithic Frontend Component

The user interface, responsible for navigating the Service Request catalog and completing the associated forms, is an excessively large and complex Vue.js frontend component (nearly 11,000 lines). This architecture makes modifications, extensions, and client-specific customizations exceptionally challenging [20]. The logic for managing dynamic fields, their interdependencies, and the rules for compilation and validation are all handled at the frontend level through a long series of hard-coded conditional statements, such as the following:

```
<div v-if="field.desc == 'Domain'">
    ...
</div>
```

This, in addition to mixing business logic with presentation, represents a significant challenge for catalog configuration, as there is no centralized way to manage the fields and their relationships. Every change, therefore, requires a direct intervention on the code, making the system inflexible and difficult to maintain, especially in a context where the operator modifying the catalog is not a programmer and does not know how the frontend code is structured. For instance, if an operator wanted to add a new "Domain" field to a Service Request, this field would need to have the exact description "Domain" and type "domain"; otherwise, the Vue.js component would not render it correctly.

These dynamics make the system fragile and susceptible to frequent errors that waste time and resources for both clients and operators [20].

Overly Generic Backend REST Service

The only information about the fields returned by the backend REST service (`SRPWebSeviceREST`), besides the field name and its mandatoriness, is its *type*. It is therefore easy to imagine how this information, which should only be an indication of the component that needs to be rendered (e.g., textbox, select, radio button, etc.), can become an identifier for something much broader.

For example, a field of type `UPN` is a composite field derived from the value of another field, '`Display Name`', concatenated with an '`@`' symbol, and suffixed with the value from a select field whose options are the client's domains, provided by an external REST service (e.g., "`n.guarini@elmec.it`", where "`n.guarini`" is the value of the "`Display Name`" field, and "`elmec.it`" is the

value of the "Domain" select field).

Despite this complexity, the REST service returns only a JSON document of this type:

```
{
    "idField": 1603,
    "descField": "UPN",
    "type": "upn",
    "mandatory": true
}
```

thus delegating to the frontend the responsibility of interpreting the type and rendering the field correctly.

Dependency on the SRP Agent

The import of data from clients' Active Directories and the actual execution of Service Requests are handled by an agent (`SRPWindowsService`) installed directly on the clients' Domain Controllers. This approach presents several challenges:

- **Security and Permissions**: The agent often requires credentials with elevated privileges (e.g., `domain-admin`), which clients are increasingly reluctant to grant.
- **Reliability and Control**: The configurations on the agent and the Domain Controller are complex and difficult to manage centrally, which can cause execution errors and data synchronization issues.
- **Scalability**: The need to install, configure, and maintain an agent for each client domain represents a significant operational effort. Alternatives such as more modern provisioning systems will be evaluated.
- **Obsolete Communication Patterns**: The unidirectional communication through the *DMI Console* creates a bottleneck and complicates the management of asynchronous operations. Communication between components in the client's network and the company network is based on polling (e.g., the SRP Agent queries the server every 5 minutes), an approach that introduces latency and inefficiencies.

1.3.2 Code and Data Management Issues

Code and data persistence management is another critical aspect of the system, characterized by a lack of versioning and modern debugging tools.

Unversioned and Undebuggable Stored Procedures

The intensive use of Stored Procedures[2] in the MSSQL database represents one of the major weaknesses. These procedures contain fundamental business logic and are:

- **Unversioned**: There is no centralized version control system for Stored Procedures. Changes are made directly on the database, making it impossible to track changes, revert to previous versions, or test modifications in separate environments.
- **Difficult to Debug**: Debugging Stored Procedures is a complex and cumbersome process, often requiring direct access and specific SQL server tools, with the risk of affecting the production environment.

Inaccessible and Unversioned SSIS Jobs

SSIS packages[3] are essential for data import and export operations (e.g., AD master data). However, these jobs reside physically on the `ELMEC-CMI-MSSQL` server, and the only way to access and modify them is via a remote desktop. This approach prevents versioning and collaboration, limits accessibility by making maintenance and debugging an onerous and risky activity, and creates a *single-point-of-failure* and a strong coupling between the import/export business logic and the infrastructure.

1.3.3 Database Schema and Data Integrity Issues

The analysis of the database schema, represented in figures 1.7, has revealed a number of significant issues that affect the quality and integrity of the data managed by the SRP system.

Lack of Data Integrity and Consistency

One of the most significant problems concerns the lack of explicit referential integrity constraints. In the schema in figure 1.7, although the relationships between tables have been inferred manually, they are not defined at the database level through the use of foreign keys. This absence prevents the

[2]Stored procedures are SQL code routines, or sets of instructions, that are pre-compiled and stored within a relational database. They act as logical units of execution, encapsulating business logic directly at the database level [37].

[3]SQL Server Integration Services (SSIS) is a platform for building data integration and workflow solutions. It is a key component of Microsoft SQL Server, primarily used for Extract, Transform, Load (ETL) operations, enabling the extraction of data from various sources, its transformation, and its loading into different destinations [31].

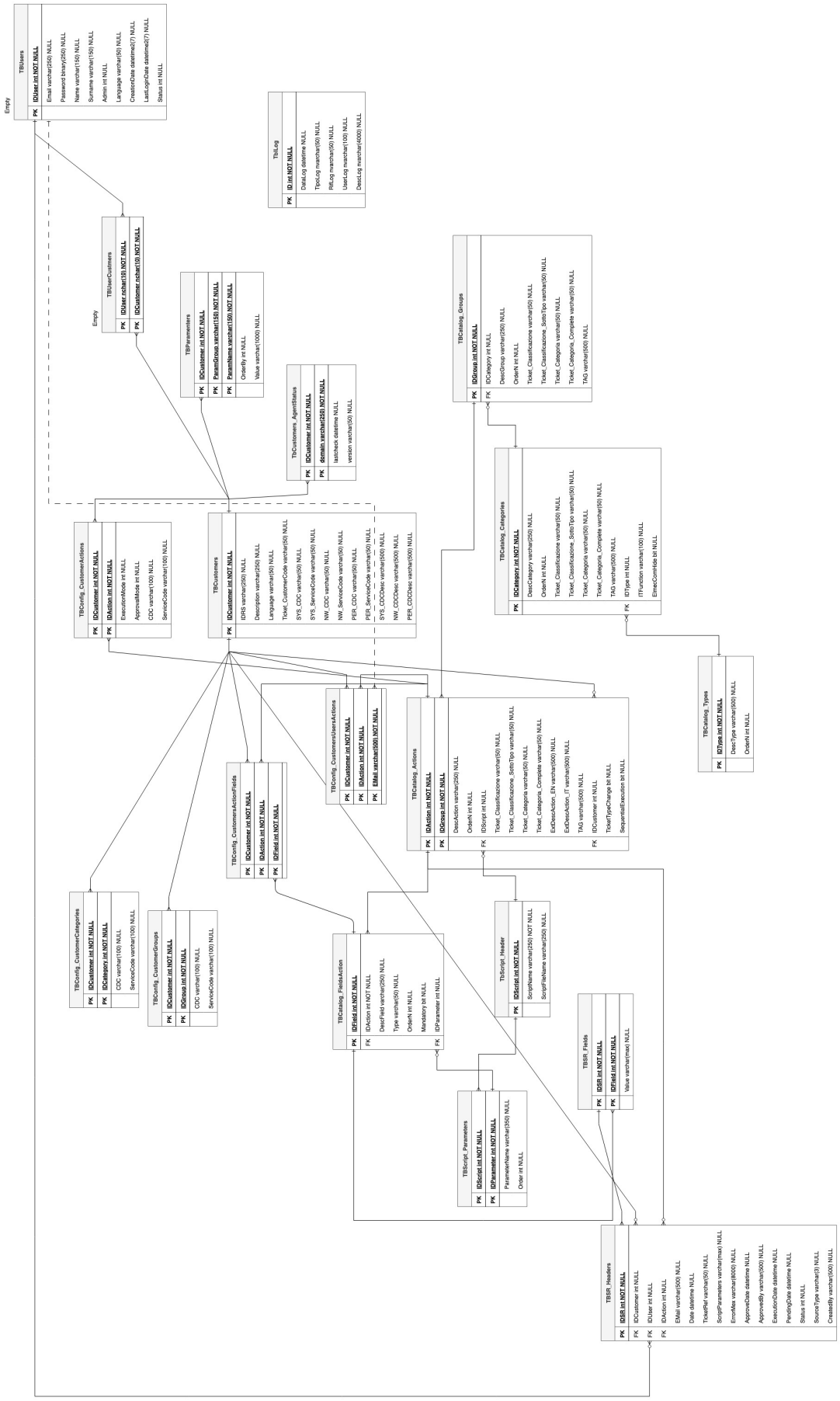

Figure 1.7: AS-IS Database ER Schema (TODO split into multiple smaller images)

database from guaranteeing data integrity [6]: for example, a row in a child table could refer to a non-existent row in the parent table [45]. Such a scenario, known as a "dangling reference[4]", can generate anomalies and inconsistencies in the system, making the data unreliable and debugging extremely complex [6].

Data Redundancy

Another issue is the high data redundancy and the large number of duplicate attributes across different tables [35]. This design, which deviates from the principles of normalization [2], leads to storing the same information in multiple locations, which not only increases the data volume but also introduces a high risk of inconsistencies [2].

Unclear and Inconsistent Naming Conventions

The naming convention for attributes and tables is inconsistent and unclear. The schema presents a mixture of conventions (e.g., `IDGroup` vs. `GroupID`, English names mixed with specific terms) and typos (e.g., `TBParamenters`), making it difficult to immediately understand the function of each field. In many cases, the naming is not explicit enough to describe the role of an attribute, requiring an in-depth knowledge of the system to correctly interpret the relationships and business logic. This lack of a unique and standardized vocabulary hinders collaboration among developers, complicates code maintenance, and increases the likelihood of errors.

1.3.4 Maintenance and Operational Challenges

According to information discussed in an internal meeting [28], the daily management of the system is made complex by several inefficiencies and obsolete design choices.

The current system design does not allow for the effective reuse of Service Requests and catalog configurations. Consequently, every modification or addition requires the creation of a new Service Request, even for small variations, leading to a innefficient and difficult-to-manage catalog, consisting of numerous duplicate SRs and fields. This, while functional, is not a scalable approach and leads to significant inefficiencies in catalog management itself. For example, if a Service Request needs to be modified for all clients who use

[4]The "dangling pointer" is a problem typically addressed in the context of low-level programming, where a pointer refers to a memory area that is no longer valid because it has been freed or because the pointer is used outside the context of the variable's existence to which it refers [15], but the same dynamic can also be applied in the context of databases, where the "pointers" are represented by foreign keys.

it, the operator will have to modify not only the SR in question but also all the duplicate SRs for each client, with the risk of forgetting a modification or making mistakes. This approach, although probably conceived to prevent error propagation, makes the catalog difficult to navigate and maintain.

The new architecture will need to address these challenges by introducing a more robust, versioned system based on modern software design principles and patterns.

Chapter 2

Architectural Redesign

Given the numerous critical issues that emerged from the analysis of the current system, it is clear that a thorough redesign is necessary. This redesign must be able to perform the system's current functions more efficiently and in an optimized manner, while also satisfying a series of requirements that did not exist when the system was developed several years ago, and which the current system cannot manage because it was not designed to do so.

2.1 New System Requirements

The system can be divided into three main components: **Catalog Structure and Management**; **Service Request Execution**, and **Master Data Import**. The requirements for each of these areas will therefore be analyzed.

2.1.1 Catalog Structure and Management

This is the most substantial component of the three, dealing with everything concerning the definition, maintenance, and configuration of the actions executable by each customer. This therefore also includes the management of form fields, customer-specific customizations, scripts, their parameters, the target machine address retrieval logic, and so on.

Before proceeding with the analysis of the requirements, a clarification on the terminology to be used is necessary: an '*Action*' is understood as the formal definition of an operation that a client can execute on its infrastructure; a '*Service Request*' (SR), on the other hand, is understood as the effective execution of an Action. If a parallel with OOP is to be drawn, the Action can be seen as a class, and the Service Request as an object, an instance of the aforementioned class.

Actions

The Action entity can be seen as the core of the catalog component, and it is composed of the following main elements:

- General metadata, such as the name, an extended description, tags, a type, a group, a category, and other information related to administrative

and billing aspects;

- The fields of the corresponding form that must be filled by the client for its execution, including the relative logic tied to types, data validation, dependencies between various fields (e.g., fields that, once completed, "enable" other fields), autofill (e.g., fields whose values are autofilled with values from other fields), sensitive values, and so on;
- The script that will be executed in the customer's infrastructure to perform the effective required operation;
- Script parameters, which may be field values, but also additional parameters necessary for execution (e.g., AWS credentials, 365 credentials, etc. . .);
- The target machine on which the script must be executed.

Some of these dynamics are already implemented in the current system (with all the critical issues of the case, as already discussed in the previous chapter), while others are not. The crucial point, however, is that all these configuration aspects must be customizable for specific customers and for specific actions [28].

Given that the users of this system are very large clients (Fendi, Valentino, Lavazza, to name a few), it is easy to imagine that each of them has highly specific needs, which make direct reuse of Actions across multiple clients impossible. At the same time, however, the creation of infinite duplicated Actions that share almost everything, with only small differences (as happens in the current system [28]), must be avoided.

It is therefore necessary to implement a system that provides for the possibility of defining a sort of hierarchical structure, where a "parent" action is configured, which can be "inherited" by several child actions that will specify only the modifications, all without affecting the default action.

Fields

Together with actions, fields are central components on which a large part of the system is based. As already mentioned, fields are entities that can be extremely diverse, such as text fields, selects, multiple-choice selects, radio buttons, etc. In the current system, there are 86 different field types. This number is so high because, due to how the current system is built, the "type" information is the only way the backend communicates to the frontend what type of field is to be rendered [34]. In fact, there are many different types that share the same logic, perhaps simply using two different services to retrieve the data. The effective field typologies are therefore much fewer, and it is crucial that they are identified and generalized as much as possible.

The general idea is that the logic for fields must be moved to the backend, which must return all the necessary information for the frontend to render the form: from field types, to validation regexes, to error messages, to any external services to be contacted to retrieve the options list, or to validate the data, and so on [28].

All this must obviously be compatible with the customization requirements specified in the previous section.

Regarding customizations, a typical example scenario could be the following: a customer uses an action, but for some reason requires two additional fields, which will then be needed by the script that will in turn require two additional parameters. Furthermore, for some fields, the client in question uses different validation rules and specific autofill templates. These customizations must be possible without modifying the default action and fields.

Regarding "particular" field types, the system must provide for [28]:

- Select fields whose options are customer-specific (e.g., "Domain" field, which is a select with all the customer's domains);
- Select fields whose options are global for the entire system (e.g., "Country" field, which provides a list of states that are the same for all customers);
- Select fields whose options are customer-specific and to obtain them an external service must be contacted, with the possible option to search (e.g., "Server" field, which is a select with all the customer's servers, returned by the Atlantis CMDB service);
- Autofilled text fields, meaning their values are obtained through a templating system (Jinja) that uses the values of other fields (e.g., "Logon Name" field, populated by concatenating the user's first and last name separated by, for example, a dot);
- Text fields whose validation occurs through regex (e.g., "Date" field, which must comply with the standard format);
- Text fields whose validation occurs through an external service (e.g., "Logon Name" field, which requires contacting the relevant Active Directory service for validation that a user with the same logon name does not already exist);
- Fields with sensitive values, whose values will therefore need to be saved encrypted, using some symmetric encryption algorithm (e.g., "Password" field);
- "Mixed" fields, composed of multiple "sub-fields" with different characteristics (e.g., "UPN" field, which is composed of the first letter of the

first name, the last name, an at sign (`@`), and the value of a select whose options are obtained from an external API call);

It is important that the system is designed in a scalable way, so that if another field typology were to be added in the future, it can be implemented with the minimum possible effort.

Configuration and Execution Consistency

A system for monitoring the consistency of the catalog configurations [28] will be necessary, in order to prevent the creation of invalid configurations, which would then lead to errors during SR execution.

For example, if a `select` type field is specified for an action, whose options are obtained from customer-specific parameters, but those parameters have not been defined for that customer, the system must report this inconsistency through some alerting system (e.g., email, ticket, monitoring dashboard). Alternatively, if an action is configured to be executed on a certain type of machine but the field necessary to select it is not among the action's fields, the system must report this inconsistency.

Similarly, a system for monitoring the consistency of SR executions must be implemented, in order to be able to report any errors that may have occurred during execution, such as an SR stuck in an "intermediate" state (e.g., `NEW`) for too long, or an SR that started execution (i.e., in `PENDING`) but never reached completion (i.e., in `COMPLETED` or `ERROR`) within a certain time limit (e.g., 2 hours).

Additional script parameters

Another topic that must be managed is that of script parameters. In the old system, there was granular control over every single parameter, which was linked to an action's script and possibly to a field in the corresponding Service Request. In practice, all the action's fields with their respective values were passed as parameters to each script, plus some additional parameters not linked to any field (e.g., Exchange Server, AD Sync Server, various credentials). This dynamic of additional parameters occurred through the definition of parameters with the foreign key on the field set to `NULL`, which were then intercepted by the stored procedure relating to the generation of the parameter string which, with hard-coded `if` statements (e.g., `IF @ParamName = 'ServerExc' BEGIN ...`), were valued accordingly, retrieving the information with custom and non-standardized logic.

In the new system, it will be necessary to standardize these dynamics, finding a way to define the additional parameters that will be needed, beyond those of the fields, during action configuration, and to generalize the implementation logic as much as possible, so that if new "groups" of additional parameters

(e.g., AWS credentials, O365 credentials, etc.) were to be added, it can be done in the most efficient way possible.

Target Machine Configuration

In the old system, Service Requests were executed exclusively on the customers' Domain Controllers. Specifically, each DC ran an agent, which periodically checked via *polling*[1] if there were any Service Requests in the `PENDING` state with a matching `domain` field.

This part of the architecture will also need to be redesigned, as it is inefficient and highly limiting by design [41]. Therefore, in addition to redesigning these dynamics, it will also be necessary to provide the option to choose, during the configuration of an Action, to execute the related SRs on other types of machines, such as internal Elmec workers, or other types of machines on the customer's network other than the Domain Controller (e.g., Exchange Server). It must also be possible to choose to execute SRs through the old flow, in order to continue supporting both systems, at least initially [28].

2.1.2 Service Request Execution

In the current system, the execution of SRs occurs according to this sequence of main events:

1. The frontend sends the request to create a new SR to the REST service (`SRPWebServiceREST`), with all the necessary data (completed fields, customer, action, etc.) [34];

2. The SR and the relevant field values are created in the database, setting the status to `NEW` [34];

3. An SSIS job, every minute, checks if there are SRs in the `NEW` state, and if any are found, they are processed, moving them to subsequent states (e.g., `WAITING FOR APPROVAL`, `PENDING`, `ASSIGNED`, etc.) depending on the action and customer configuration [29];

4. When the SR is in the `PENDING` state, it means it is ready to be executed by the agent on the customer's DC;

5. Every 10 seconds, the agent on the customer's DC checks if there are SRs in the `PENDING` state for its domain (through calls to the SOAP service (`SRPWebService`)) [35];

[1] *Polling* is a communication technique where a system or device periodically and actively samples the status of an external resource or device to check for readiness or new data.

6. If any are found, for each one, the corresponding script is downloaded, executed, and the SR status is updated to `COMPLETED` or `ERROR` depending on the execution outcome (through the SOAP service) [35];

It has already been discussed in the previous chapter how this flow has numerous criticalities, which make it inefficient and not very scalable. It will therefore be necessary to completely redesign it, taking into consideration the possibility of executing SRs on machines other than DCs, and the possibility of executing them also on machines internal to Elmec, and even virtual machines [28].

Elmec already has an internal *provisioning* system, called *Provision Prime*, which is responsible for executing Ansible playbooks on machines (internal, external, and also virtual) [24], and which could be leveraged for this purpose. It will therefore be necessary to analyze this system, and understand if and how it can be integrated into the new SR execution flow, in order to avoid the polling of agents on the DCs, and to have a generally more reliable and efficient system [28].

A constraint that must be respected, however, is that the action scripts are all in PowerShell, and it is not possible to modify them. This is because they are very numerous, have been developed over the years, have been tested and validated for every single client, and are maintained by different Elmec departments. Therefore, even if the SR execution flow is redesigned, the scripts must be executed as they are. This obviously does not apply to new scripts that will be developed in the future, which can be designed leveraging the new technologies [28].

2.1.3 Master Data Import

Another important component of the system is the one responsible for importing customer master data, necessary for populating some action fields (e.g., UPN domains, user accounts, groups, Organizational Units (OU)[2], databases, etc.). The main difficulty lies in the fact that this information physically resides on the customers' machines (large enterprise clients, with thousands of user accounts and groups, are being discussed), and therefore their retrieval in real-time is not immediate.

In the current system, this import occurs in a manner very similar to the SR scenario, but in reverse. It happens through an agent running on the customers' DCs, which periodically (hourly) (downloads and) executes a series

[2]In Windows Server, an Organizational Unit (OU) is a container within Active Directory (AD) used to logically group objects like user accounts, computer accounts, and security groups, similar to a folder in a file system [1].

of PowerShell scripts to retrieve the necessary information, and deposits them in a specific system folder (`D:\SRPortal\Anag\`) [35]. These files are then consumed by an SSIS job (one for each type of master data) that imports them into the database. The REST services then expose APIs to retrieve this information, which are used by the frontend to populate the action fields [34].

In the new system, it will be necessary to redesign this flow as well, in order to eliminate the dependence on agents on customer DCs. Even in this case, however, the PowerShell scripts cannot be modified [28], and must be executed as they are now. It will therefore be necessary to analyze how to integrate the execution of these scripts into the new flow, perhaps also leveraging Elmec's internal *provisioning* system (*Provision Prime* [24]) in this case, in order to obtain a more traceable, reliable, and efficient system [28].

2.2 New System Architecture

After gathering and defining the requirements for the new *ServiceRequestManager (SRM)* system through various meetings with stakeholders and analysis of the current system (SRP), it has been possible to design a new architecture for the system that allows for efficient and scalable satisfaction of these requirements.

The proposed new architecture for the SRM system, shown in Figure 2.1, is based on a **microservices architecture**, which allows for greater modularity, scalability, and ease of maintenance compared to the distributed-monolithic architecture of the current SRP system [10].

2.2.1 Main Components

The SRM system architecture is composed of the following main components.

Frontend

Two separate web frontends will be developed for end-users and system administrators. These frontends will communicate with the REST services via HTTP/HTTPS APIs.

Due to the business logic being moved to the REST services, the frontends will be lighter and easier to maintain, allowing for greater flexibility in implementing new functionalities and user interface improvements [20]. This also allows for the future integration of a mobile application, which can use the same APIs.

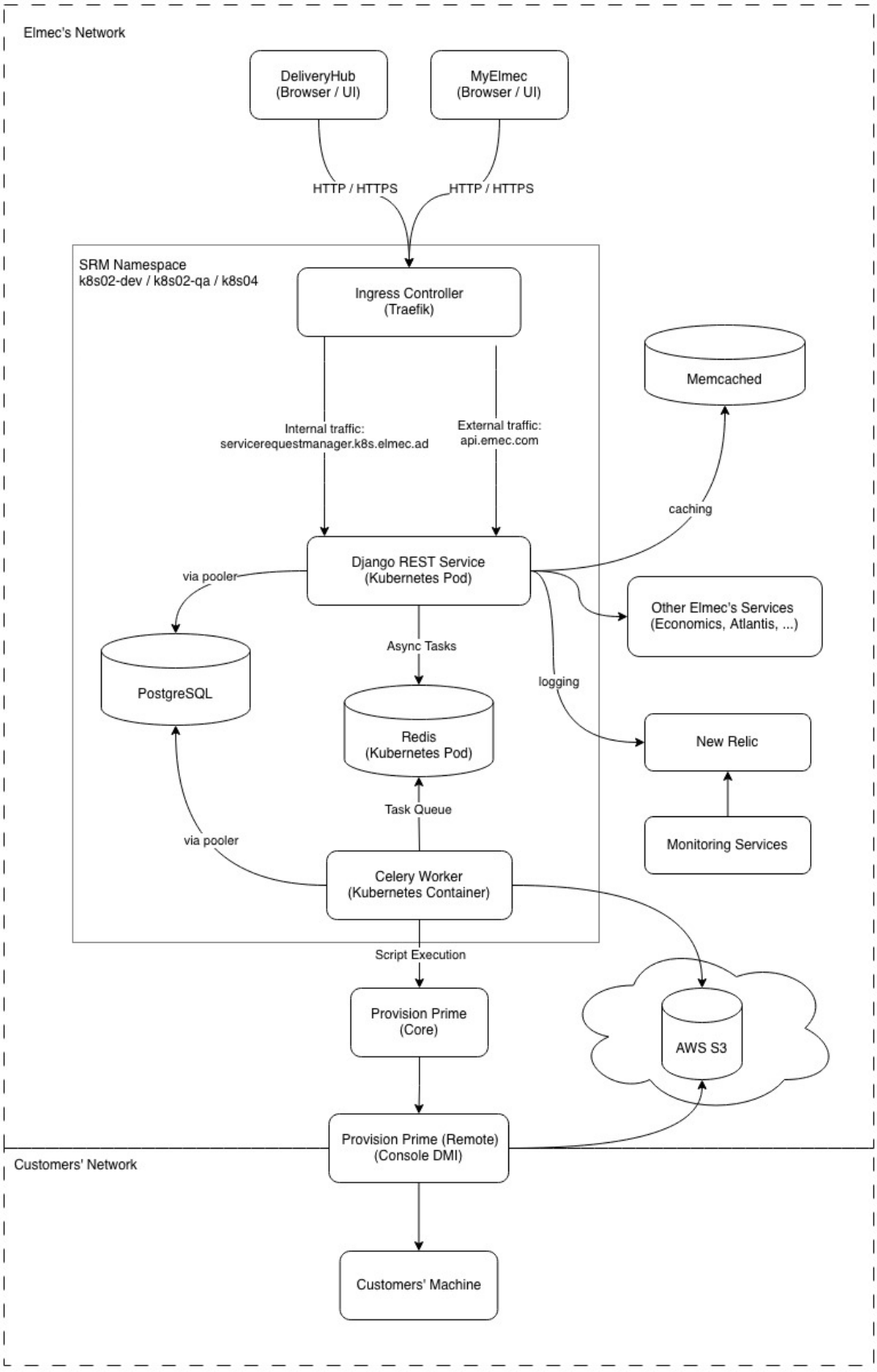

Figure 2.1: SRM System Architecture

REST Services

REST (Representational State Transfer) is an architectural style for designing web services that leverages HTTP protocols for communication between client and server [14]. The basic idea is that a system's resources (users, orders, documents, etc.) are exposed through a uniform interface, typically HTTP, using [14]:

- URLs to identify resources (e.g., `/api/catalog/actions/123/`);
- HTTP verbs to operate on resources (`GET`, `POST`, `PUT`, `DELETE`, ...);
- Representations of resources in standard formats (JSON, XML, ...).

These services will be responsible for business logic, user request management, and interaction with databases and other system components.

Authentication and Authorization System

Authentication and authorization will be managed through the internally developed library, PyElmecAuth or GoElmecAuth (depending on the implementation language), which will interface with the corporate identity management system to ensure secure and controlled access to the REST services.

It will therefore be necessary to integrate a granular permission system into SRM to ensure that users can only access the resources and functionalities for which they are authorized, based on the roles defined by the corporate Identity Provider.

Caching System

A caching system will be required to improve system performance and reduce the load on the databases.

In particular, responses from calls to external services will be cached, such as third-party APIs needed to validate some fields, or calls to retrieve customer information, to reduce response times and improve the user experience. The time-to-live (TTL) of the caches must be configurable based on the specific needs of each data type.

Message Broker and Task Queue

To manage asynchronous operations and improve system scalability, a Message Broker (e.g., RabbitMQ, Apache Kafka, Redis, ...) will be used in combination with a Task Queue (e.g., Celery).

This is necessary as the SR lifecycle is very complex and involves many operations that can require long execution times [27], such as sending emails,

maintaining HDA[3] tickets, integration with external systems, etc. By using a messaging and task queue system, these operations can be executed in the background, without blocking the application's main flow [38].

Provisioning System

It will also be necessary to integrate a provisioning system to execute the scripts related to SRs and those for master data import directly within the clients' networks. This system must be secure, reliable, and scalable, in order to be able to manage a large number of provisioning requests simultaneously.

Elmec already has its own provisioning system called *Provision Prime* [24], which will be integrated with the new SRM system to execute scripts efficiently and securely.

Monitoring and Logging System

To ensure the stability and reliability of the system, a monitoring and logging system will be required. This system will allow tracking system performance, identifying any problems, and analyzing logs to resolve bugs. Tools such as Prometheus, Grafana, New Relic, or other similar tools will be used to implement this system.

Both system metrics (CPU, memory, database usage, API response times, etc.) and application logs (errors, warnings, debug information, etc.) will be tracked, in order to always have a complete view of the system status, and to have the possibility of launching automatic alarms in case of problems (e.g., if the number of responses resulting in 500 (Internal Server Error) in the last hour is $\geq 1\%$).

Containerization and Container Orchestration System

To facilitate system deployment, scalability, and management, a containerization system (Docker) will be used in combination with a container orchestration system (Kubernetes).

This will allow isolating the different system components, simplifying the deployment process, and ensuring greater resilience and scalability of the system, through functionalities such as replicas, chronjobs, automatic scaling of pods, and so on [18].

[3]HDA (Help Desk Advanced) is a web-based ticketing and service management system developed by Zucchetti. It enables organizations to manage tickets, incidents, and support processes through a centralized platform [16].

2.3 Addressing the Identified Challenges

All the challenges described in section 1.3 have been analyzed, addressed, and resolved through a series of targeted interventions. These interventions concerned various aspects of the system, from restructuring the software architecture and revising data management processes to implementing new technologies and development practices. The solutions adopted for each of the identified challenges are detailed below.

2.3.1 Architectural and Technological Challenges

The adoption of an approach based on containerization (with Docker) and orchestration (with Kubernetes, often abbreviated as K8s) forms the foundation for overcoming the current architectural and technological limitations of the SRP system. This transition not only directly addresses issues of modularity, isolation, and dependency management but also paves the way for a more modern and scalable architecture, such as microservices or micro-frontends. Specifically, it aims to solve the problems of fragmentation, dependency complexity, and maintenance difficulties inherited from the previous monolithic architecture and the agent installed at the customer's site.

Containerization with Docker

Docker is a containerization platform that allows developers to package applications and all their dependencies (libraries, configurations, environments) into a single, standardized unit called a container. As illustrated in figure 2.2, these containers are isolated from each other but share the host operating system's kernel, making them extremely lightweight and portable.

The use of Docker resolves the "works on my machine but not in production" problem (known as "dependency hell") [3] and provides a solid foundation for component decoupling. Each component (frontend, backend, support services) will be enclosed in a *Docker container*, and ensures that the execution environment (including libraries, configurations, and dependencies) is identical across development, testing, and production [3]. This eliminates errors caused by environmental discrepancies.

Once created, the container can be run anywhere Docker is present (in our case, container orchestration is managed by Kubernetes, which will be discussed in the next subsection), simplifying the deployment process. The system thus becomes extremely more portable, moving away from the strict dependence on specific operating systems and hardware configurations like, in this case, Windows Server and Microsoft SQL Server.

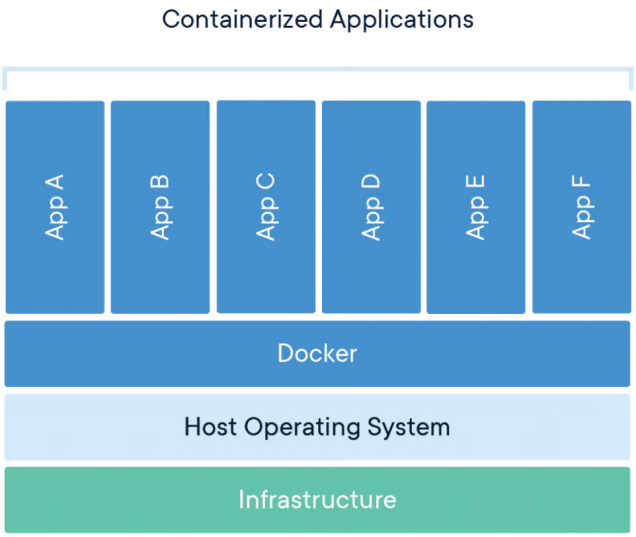

Figure 2.2: Docker Container Architecture (source: [44])

Orchestration with Kubernetes

Kubernetes (K8s) is an open-source system for automated container orchestration, originally developed by Google. Its purpose is to manage entire fleets of Docker containers in a *cluster*, ensuring that applications are always available, scalable, and resilient [18].

If Docker provides the mechanism for creating and distributing containers, Kubernetes is its natural complement, handling their large-scale management (*orchestration*), transforming the infrastructure from a set of single machines into a centrally managed cluster [18].

K8s allows components to scale horizontally based on the workload (via the Horizontal Pod Autoscaler). When demand increases, K8s automatically launches new instances of services (e.g., more pods for the REST backend) and removes them when the load decreases. This is crucial for managing traffic peaks [26].

In parallel, it constantly monitors the status of the containers. If a component fails, K8s automatically restarts it or schedules a new one on a healthy node in the cluster, ensuring high availability and resolving the single-point-of-failure problem introduced by the current dependence on single servers and the agent [26].

Kubernetes also supports advanced deployment strategies (e.g., Rolling Updates), allowing code updates without service interruptions (zero-downtime deployment) [26]. In case of issues, rolling back to the previous version is fast

and automated. This mitigates the risk associated with system changes.

Finally, K8s also offers native mechanisms to manage environment variables and sensitive data (such as database credentials or external service authentication tokens) separately from the container code [26]. This centralizes management and increases security, eliminating the numerous hard-coded configurations present in the old system and making the containers more generic.

The adoption of Docker and Kubernetes therefore creates the ideal environment for the subsequent decomposition of the frontend monolith and the re-engineering of the various backend components, topics that will be covered in detail in the following sections.

Monolithinc Frontend Component

At a technological level, the chosen technology did not change compared to the existing system, which is Vue.js. However, Vue.js version 3 was used, which introduces numerous improvements compared to version 2 used in the existing system, including better performance management, a more flexible composition system, and improved support for TypeScript [43]. In fact, in addition to the updated version of Vue.js, it was decided to adopt TypeScript as the development language for the frontend, in order to improve code maintainability and quality [40].

TypeScript Developed in 2012 by Microsoft as free and open-source software (Apache License 2.0) [40], TypeScript extends JavaScript's functionalities by introducing optional static typing. The main advantage is that, unlike JavaScript, where type errors are detected only at runtime, TypeScript allows most errors to be caught already at the *build time* stage. This ensures greater code robustness and a significant reduction in production bugs. For extended projects like this one (the Service Requests component is actually only a part of a much larger customer-dedicated platform called *MyElmec*), static typing significantly improves code maintainability and readability, acting as a form of implicit documentation and allowing for safer and faster refactoring.

To solve the maintainability and scalability problems of the existing monolithic frontend, it was decided to move all the configuration and customization logic for field behavior (autofill, dynamic fields via external APIs, validation, dependencies, etc.) to the backend (a topic that will be explored in section 2.3.1). In this way, the frontend was transformed into a simple interface for display and user interaction (as it should be), significantly reducing code complexity and facilitating the implementation of new functionalities. In particular, the frontend is now limited to reading a JSON configuration provided by the backend, which describes the form structure, the fields to be displayed, their properties, and validation rules. This was also made possible thanks to the

potential offered by Vue.js 3, which allows for the creation of highly dynamic and reusable components based on external data [43].

This led to a transition from a total of 86 different "types" of fields in the existing system to only 10 generic field types:

- Text
- Numeric
- File
- Password
- Checkbox
- Select (with already defined options)
- Select (with API-fetched options)
- Date / DateTime
- UPN (managed specifically due to its peculiarities, as it is composed of two sub-fields and has a particular validation logic)
- ADOU Tree

All the various customization logics (especially the customer-specific ones) thus become transparent to the frontend, which merely interprets the configuration provided by the backend. The system transitioned from a component with approximately 11,000 lines of code to a component with only 1,000 lines of code, resulting in improved maintainability and ease of system extension, and with the possibility of implementing other frontends (e.g., a mobile app) with minimal effort without having to replicate all the complex field management logic.

Overly Generic Backend REST Services

To solve the issues related to the frontend, a deep restructuring of the backend logic was necessary, especially concerning fields, moving from JSON responses of this type

```
{
    "idAction": 455,
    "idField": 1649,
    "descField": "Manager",
    "type": "ADUser",
    "idCustomer": 42,
    "orderN": 53,
```

```
    "mandatory": false
}
```

to much more detailed and specific responses, which provide the frontend with all the necessary information for the complete rendering of the entire form, such as the following:

```
{
    "field": {
        "id": 3001,
        "name": "manager",
        "description": "Manager",
        "type": "select",
        "is_sensitive": false,
        "option_source_policy": "EXTERNAL_API",
        "external_api_config": {
            "name": "AD_USERS",
            "base_url": "https://q-api.elmec.com/srp/",
            "endpoint_pattern":
             ↪  "api/customer/VFG/adusers?domain={{domain}}",
            "http_method": "GET",
            "headers_json": {
                "Authorization": "{{TOKEN_TYPE}} {{TOKEN}}"
            },
            "response_mapping_json": {
                "name": "userDesc",
                "value": "user"
            },
            "timeout_seconds": 10,
            "is_searchable": true,
            "search_param_name": "search",
            "root_parent_id": null
        },
        "options": null,
        "multiple_choice": false,
        "file_accept_mimes": null
    },
    "mandatory": false,
    "validation_regex": "^[a-zA-Z0-9._]{5,20}$",
    "validation_regex_message": "Invalid format for Manager
     ↪  field.",
    "autofill_template": null,
    "depends_on": [
        "domain",
        "first_name",
        "last_name"
```

```
38      ],
39      "display_order": 26
40  }
```

In addition to modeling all possible properties of a field in a clear and structured way, the new REST services are also responsible for managing all the customization and configuration logic of the fields, with a three-level structure (which will be detailed in the chapter, dedicated to implementation aspects):

1. Fields
2. Actions
3. Customers

Broadly speaking, the idea is that each field (first level "Fields") has a set of generic properties (type, description, options, etc.) that can be further customized based on the specific action (second level "Actions") to which the field belongs (e.g., making it mandatory or not, adding validation rules, autofill logic, dependencies on other fields, etc.). Finally, each action can be further customized per customer (third level "Customers"), allowing the field's behavior to be adapted to the specific needs of each customer (e.g., modifying the available options in a selection field, changing validation rules, etc.), all without affecting the "original" configuration of the underlying level.

This three-level structure allows for a high degree of reusability and modularity, making it possible to define generic fields that can be easily adapted to different situations without having to duplicate code or logic.

Dependency on the SRP Agent

The dependency on the agent installed at the customer site has been completely eliminated thanks to the integration of Provision Prime, an internally developed provisioning tool written in Go, which manages the execution of Ansible playbooks on remote machines, in this case, the servers within customer networks, transparently to the developer (who will simply configure the playbook and make an API call).

Ansible Ansible is an open-source, agentless IT automation engine used for configuration management, application deployment, intra-service orchestration, and provisioning [4]. It is designed to be simple, powerful, and easy to use, operating over standard SSH for Linux/Unix and WinRM for Windows, thus requiring no special client software (agents) to be installed on the managed nodes [4]. This reliance on existing transport protocols and its human-readable structure, written in YAML, contribute to its minimal learning curve and rapid adoption. The core of Ansible's functionality are

Playbooks. A Playbook is essentially a blueprint of automation tasks, structured as a YAML file, that describes a set of steps to be executed on specified hosts or groups of hosts (defined in an inventory) [4]. They define the desired state of a system in a clear, declarative manner. Each Playbook consists of one or more plays, and each play is an ordered list of tasks. A task is a call to an Ansible module, which is the unit of code that performs a specific action, such as installing a package, copying a folder, or executing scripts. Playbooks ensure that complex, multi-tier application deployments and job executions are repeatable and reusable, transforming manual, error-prone processes into reliable, version-controlled automation.

In this way, not only is the dependency on the agent, and all the associated logic, eliminated, but all the scripts related to the Service Requests, which previously resided physically inside the customers' Domain Controllers and were not tracked in any way, are now versioned.

The cost of these improvements is represented by the additional logic needed to manage communications with Provision Prime, specifically the retrieval of the target machine. To implement these retrieval logics, which are highly custom for each type of target machine (e.g., Domain Controller, Exchange Server, internal workers, etc.), the *Strategy* design pattern [25] was leveraged. This pattern allows each retrieval logic to be encapsulated in a separate class, making the code more modular, extensible, and maintainable (a topic that will be explored in depth in Section **??**).

As for the script execution logs, which are needed to monitor the status of Service Requests and detect execution errors, an integration with an AWS S3 bucket, a scalable and durable storage service, will be implemented. This way, every time a script is executed on a remote machine, the logs will be uploaded by the DMI (via the Ansible playbook) to the S3 bucket, ensuring centralized and secure access to execution logs regardless of the machine on which they were generated.

2.3.2 Code and Data Management Issues

The lack of versioning and modern debugging capabilities outlined in the existing system is fundamentally addressed by centralizing all disparate logic (previously scattered across Stored Procedures and SSIS jobs) into a unified, version-controlled software project integrated with a robust CI/CD pipeline. By adopting Git as the standard Version Control System (VCS), all business logic is now treated as application code. This immediately solves the versioning problem: every change is tracked, attributed to an author, and can be reverted, enabling seamless collaboration and a full audit trail. Moreover, moving the core business logic out of the database and into a dedicated backend service (Django REST Framework) written in a modern language (Python) drastically improves debuggability, allowing developers to use powerful, standardized IDE

tools to set breakpoints, inspect variables, and step through code without impacting the production database.

This strategy is completed by integrating several specific CI/CD pipelines (a topic that will be explored in depth in Section ??) tailored for each environment branch (`feature`, `devel`, `quality`, `main`). This setup automates the deployment process: code changes are first pushed to a dedicated task branch, reviewed, and then merged into the `devel` environment. Upon passing, the code is merged to the `quality` environment, and finally to `main` for production release. This process ensures that no logic is directly modified in a production environment, eliminating the single-point-of-failure inherent in direct database and server access. Furthermore, replacing the inaccessible, unversioned, and tightly coupled SSIS Jobs with modern, containerized async tasks (managed as code within the Git repository) breaks the strong infrastructure coupling, enhancing accessibility, facilitating collaboration, and making maintenance and debugging a safe, environment-specific activity.

2.3.3 Database Schema and Data Integrity Issues

The issues related to database design and data management were addressed through a series of interventions aimed at improving data integrity, consistency, and maintainability. These interventions include a complete revision of the database schema to eliminate redundancies and inconsistencies, the implementation of stricter integrity constraints, and the adoption of clear and consistent naming conventions for tables and fields.

The system transitioned from using Microsoft SQL Server as the Database Management System (DBMS) to PostgreSQL, an open-source DBMS known for its robustness, scalability, and compliance with SQL standards. PostgreSQL offers advanced features such as support for complex data types, full ACID transactions, and an extensibility system that allows new functionalities to be added via extensions [36]. This choice not only improves the system's performance and reliability but also facilitates data management and the implementation of database design best practices.

Lack of Data Integrity and Consistency

To resolve data integrity and consistency issues, stricter referential integrity constraints were implemented using well-defined primary and foreign keys, also specifying behaviors in case of deletion (`ON DELETE CASCADE` / `SET_NULL`, etc...).

Specifically, for the deletion of "core" entities (such as customers, actions, or fields), the `CASCADE` behavior was chosen for "child" entities, which are consequently deleted.

For secondary entities and/or those that can be null (such as ticket categorization information, action categories and groups, order information associated with tickets, etc...), the `SET_NULL` behavior was chosen to preserve the history of operations and associated data, preventing the loss of more important information.

These constraints ensure that relationships between tables are correctly maintained, preventing the creation of orphaned or inconsistent data.

Data Redundancy

New relationships were created to maximize data reuse and reduce redundancy, while ensuring referential integrity through the use of well-defined primary and foreign keys.

These new entities primarily model concepts that were previously duplicated across multiple tables, such as cost centers (CDC, *"centro di costo"*), service codes, and ticket classification and category. This significantly reduced the amount of redundant data, improving storage efficiency and database maintainability.

Unclear and Inconsistent Naming Conventions

Now, all tables and their attributes follow standardized naming conventions (*snake_case*), improving the readability and maintainability of the database schema.

For "core" relationship names, the plural form was chosen (e.g., `actions`, `service_requests`, `customers`, etc...), while for "child" relationships, the singular name of the "parent" relationship followed by the specific name in the plural was chosen (e.g., `action_fields`, `customer_actions`, etc...).

2.3.4 Maintenance and Operational Challenges

With the new improvements introduced in this chapter concerning architecture, technology, the completely redesigned catalog and configuration management, and the new execution flow, many of the previously encountered maintenance and operational problems have been resolved. The standardization of development and deployment processes, the adoption of DevOps practices, and the implementation of a more modern and scalable infrastructure have led to a significant reduction in downtime, while improving the system's overall reliability. Furthermore, the adoption of advanced monitoring and logging tools allows for rapid identification and resolution of any issues, ensuring smooth and continuous system operation.

These improvements not only facilitate daily operations but also long-term

planning, enabling the system to adapt easily to future needs and challenges.

Chapter 3

Implementation of a modular and scalable solution

While the previous chapter described in detail the requirements and the new architectural design for the new ServiceRequestManager (SRM) system, proposing targeted solutions to overcome the various technological and structural limitations of the legacy SRP system, this chapter marks the transition from the theoretical plan to practical realization, focusing on the concrete implementation of a modular and scalable solution.

The primary focus of this implementation lies in how the critical challenges described in the previous chapters were resolved. Specifically, it will be highlighted how the centralization of field management and configuration logic (and more generally of the catalog) on the backend eliminated the monolithic frontend and enabled a highly reusable three-level configuration system. The integration of the new execution and retrieval flow for master data through a message queue system will also be illustrated, which is essential for eliminating the dependency on the obsolete SRP Agent.

3.1 Automation of the Development Lifecycle through CI/CD Integration

The project implements a comprehensive CI/CD pipeline that automates code quality verification, security scanning, testing, building, and deployment across three distinct environments: `devel`, `quality`, and `production`. The Git workflow requires feature branches to merge sequentially through these environments, ensuring progressive validation before production release.

3.1.1 Pipeline Architecture

The CI/CD pipeline is organized into three distinct workflows based on branch type, as illustrated in Figures 3.1, 3.2, and 3.3.

Feature Branch Pipeline

For feature branches, the pipeline executes automated checks for code quality, security, and compliance without building or deploying the application. The

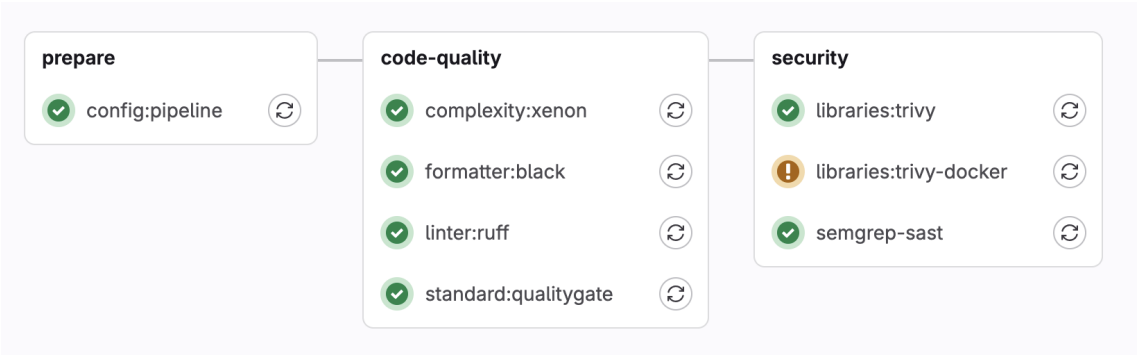

Figure 3.1: Feature branch pipeline workflow with quality and security checks

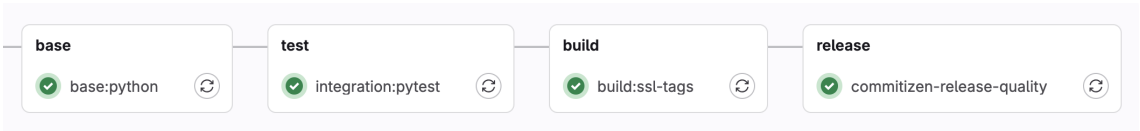

Figure 3.2: Live environment pipeline workflow with build, test, and release stages

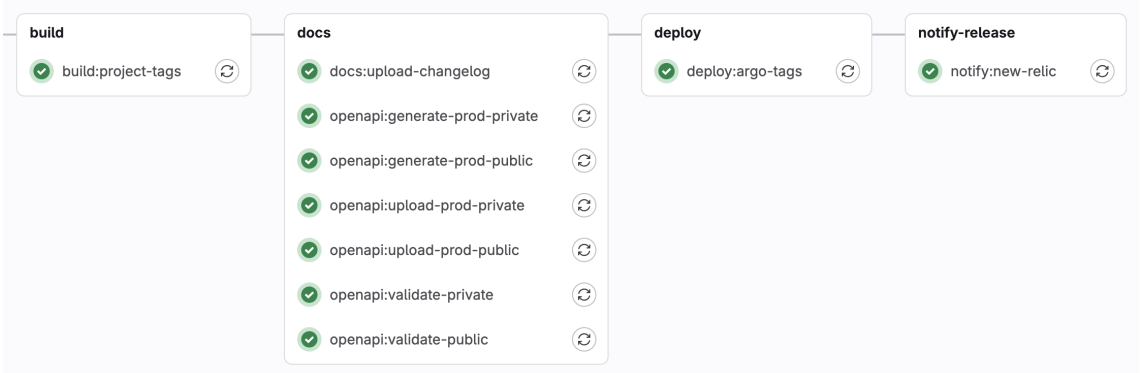

Figure 3.3: Tag-based deployment pipeline with documentation and ArgoCD integration

`complexity:xenon` stage measures code complexity through the Xenon static analyzer [46], identifying functions that might be difficult to maintain. The `formatter:black` stage verifies Python code compliance with PEP-8 conventions [21], while `linter:ruff` performs static analysis to detect errors and unsafe constructs.

Security scanning occurs through two Trivy stages: `libraries:trivy` identifies known vulnerabilities in dependencies against CVE databases [39], while `libraries:trivy-docker` scans deployment files and Kubernetes manifests for misconfigurations. The `SAST` stage uses Semgrep to detect security vulnerabilities such as SQL injection and XSS without code execution [30].

Live Environment Pipeline

For main branches (`devel`, `quality`, `master`), the pipeline includes additional stages shown in Figure 3.2. The `base:python` stage builds the base Docker image containing Python dependencies, implementing layer separation to optimize build times through intelligent cache usage. The `test:pytest` stage executes the automated test suite against PostgreSQL 16, enforcing an 80% code coverage threshold.

The `commitizen-release` stage manages automatic versioning using Commitizen, which analyzes commit history to determine version numbers according to Semantic Versioning and Conventional Commits conventions. This automated approach eliminates manual version management and ensures consistency in the release process.

Tag-Based Deployment Pipeline

When a semantic version tag is created, the pipeline triggers the workflow shown in Figure 3.3. The `build:project-tags` stage assembles the final Docker image from previously built layers and publishes it to the company registry. Documentation stages generate OpenAPI/Swagger specifications using Django Spectacular and publish changelogs to the internal documentation platform.

The `deploy:argo-tags` stages manage automatic deployment through ArgoCD [5], a GitOps tool that synchronizes Kubernetes cluster state with Git-declared configurations. Listing 1 shows the deployment trigger structure.

The `notify:new-relic` stage registers deployments in New Relic APM, creating markers that correlate software versions with performance metrics and enabling rapid identification of deployment-related issues.

```yaml
deploy:argo-tags:
  stage: deploy
  only:
    - /^\d+\.\d+\.\d+$/  # Semantic version tags only
  script:
    - candymand deploy --namespace servicerequestmanager
                       --version $CI_COMMIT_TAG
```

Listing 1: ArgoCD Deployment Configuration for GitOps-based Releases

3.2 Implementation of the Deployment Architecture on Kubernetes

The SRM deployment architecture on Kubernetes ensures scalability, resilience, and separation of responsibilities through distinct deployments optimized for specific system functions. The high-level structure shown in Listing 2 defines the namespace, service accounts, deployments, and scheduled jobs.

```yaml
namespace: servicerequestmanager
serviceAccount:
  create: TRUE
deployments:
  - name: static    # Nginx for static content
  - name: backend   # Django API + Celery workers
jobs:
  - name: sync-srp-catalog        # Daily at 01:00
  - name: sync-srp-sr-history     # Daily at 02:00
  - name: import-directory-data   # Daily at 02:00
```

Listing 2: High-Level Kubernetes Deployment Structure for SRM

3.2.1 Backend Application Deployment

The `backend` deployment constitutes the architecture core, managing application logic through multi-container organization. The deployment integrates HashiCorp Vault [42] for secrets management, automatically injecting credentials into the pod filesystem at `/vault/secrets/secrets` without exposing them in environment variables.

The `api` container runs Django through Gunicorn with Burstable QoS [23] (400m-800m CPU, 1000Mi-1200Mi memory), while the `celery` container executes asynchronous workers with identical resource allocation. Listing 3 shows the worker configuration.

The deployment exposes two services: `api` for internal cluster access and `api-ext` for public access through `api.elmec.com/servicerequestmanager/`, with path rewriting middleware removing the prefix. CORS policies permit cross-origin requests supporting third-party integrations.

```
- name: celery
  image: "docker.elmec.com/.../servicerequestmanager:VERSION"
  command: "/bin/sh"
  args:
    - -c
    - . /vault/secrets/secrets; cd servicerequestmanager;
      celery -A servicerequestmanager worker --loglevel=info
             --pool threads
  envs:
    - name: CELERY_QUEUE
      value: "servicerequestmanager_production"
    # ...
```

Listing 3: Celery Worker Container Configuration in Kubernetes

3.2.2 Scheduled Synchronization Jobs

Kubernetes CronJobs maintain system alignment during the SRP-SRM coexistence period. The `sync-srp-catalog` job executes daily at 01:00 AM, importing catalog configurations from the SRP SQL Server database. The `sync-srp-sr-history` job runs at 02:00 AM, synchronizing service request execution history. The `import-directory-data` job triggers Ansible playbooks for master data export from customer Domain Controllers.

3.3 Backend Service Implementation

The backend service is the most critical component of the SRM architecture, responsible for managing application logic, processing service requests, and integrating with external systems. Its implementation was designed to ensure scalability, resilience, ease of maintenance, and a high degree of customization to accommodate the specific needs of clients.

3.3.1 Technological Stack

The backend leverages several key technologies that form the foundation of the system architecture.

Core Framework

The backend is built on *Django* 5.2 with *Django REST Framework* (DRF) 3.16, implementing a Model-View-Controller pattern where Django models handle data persistence, DRF viewsets manage business logic and request processing, and serializers handle data validation and transformation [11][12]. This architecture was selected over alternatives like Go with GORM primarily due to development velocity considerations: Django's comprehensive ecosystem of built-in components (ORM, authentication, admin interface) significantly reduces time-to-market compared to implementing equivalent functionality in

Go. While Go offers superior raw performance, the anticipated load characteristics of SRM (primarily CRUD operations and orchestration) do not justify trading ecosystem maturity for performance gains.

Asynchronous Processing

Celery with *Redis* as the message broker implements the asynchronous task queue for service request execution and scheduled operations [7]. This producer-consumer architecture enables Django to publish tasks to Redis while dedicated worker processes consume and execute them independently, preventing long-running operations from blocking API responses. Celery was configured with thread-based workers optimized for I/O-bound operations (API calls, database queries, remote script execution coordination) rather than CPU-intensive processing.

Data Layer

PostgreSQL serves as the primary data store, selected for its robust ACID transaction guarantees, sophisticated query planning, and support for partial indexes and Common Table Expressions (CTEs) essential for hierarchical catalog queries. The application connects through *PgBouncer*, a connection pooler that maintains persistent database connections and enables horizontal scaling of application pods without exhausting database connection limits [8].

Memcached implements distributed in-memory caching [19], specifically for responses from external financial and administrative systems (contracts, cost centers, project codes, server informations, ...). These external services exhibit high latency but return data with temporal stability, making them ideal candidates for caching with configurable TTL values.

Remote Execution

Ansible orchestrates service request execution on customer infrastructure through agentless SSH/WinRM connections [4], eliminating the dependency on the legacy SRP Agent. Custom playbooks encapsulate provisioning workflows that currently wrap existing PowerShell scripts with a migration path toward native Ansible modules.

Object Storage

S3-compatible object storage (*Ceph RGW* via MinIO API) manages master data exports and execution artifacts. Master data is exported to JSON in S3, then materialized in PostgreSQL for efficient querying, achieving sub-second response times compared to multi-second LDAP query latencies.

3.3.2 Application Architecture

The Django project is organized into six specialized applications following separation of concerns principles: `catalog/` manages the three-level configuration hierarchy; `integration/` handles external system integrations; `execution/` implements service request lifecycle management; `monitoring/` provides health checks; `directory_data/` manages master data import; and `common/` contains shared utilities.

3.3.3 Catalog Management System

The catalog implements a three-level configuration hierarchy that eliminates the monolithic field duplication of the legacy system while providing granular customization capabilities.

Configuration Hierarchy

As shown in Figure 3.4, the architecture consists of three distinct levels:

- Level 1 defines global `Action` and `Field` models with default configurations.
- Level 2 provides `CustomerAction` for customer-specific action modifications.
- Level 3 offers `CustomerActionField` for granular field-level customer-specific customization.

Configuration resolution follows strict precedence: `CustomerActionField` overrides `ActionField`, which overrides `Field`, while `CustomerAction` overrides `Action`. Listing 4 shows the implementation of the target type enumeration that determines execution routing.

```python
class ActionTargetTypes(Enum):
    DOMAIN_DC = "From SR Domain (Domain Controller)"
    DOMAIN_EXC = "From SR Domain (Exchange Server)"
    INTERNAL_WORKER = "Internal Worker"
    SRP = "SRP Agent (Domain Controller)"

    def get_class(self) -> BaseActionTargetType | None:
        match self:
            case ActionTargetTypes.DOMAIN_DC:
                return DCFromDomainTargetType()
            # ...
```

Listing 4: Action Target Type Enumeration Implementation

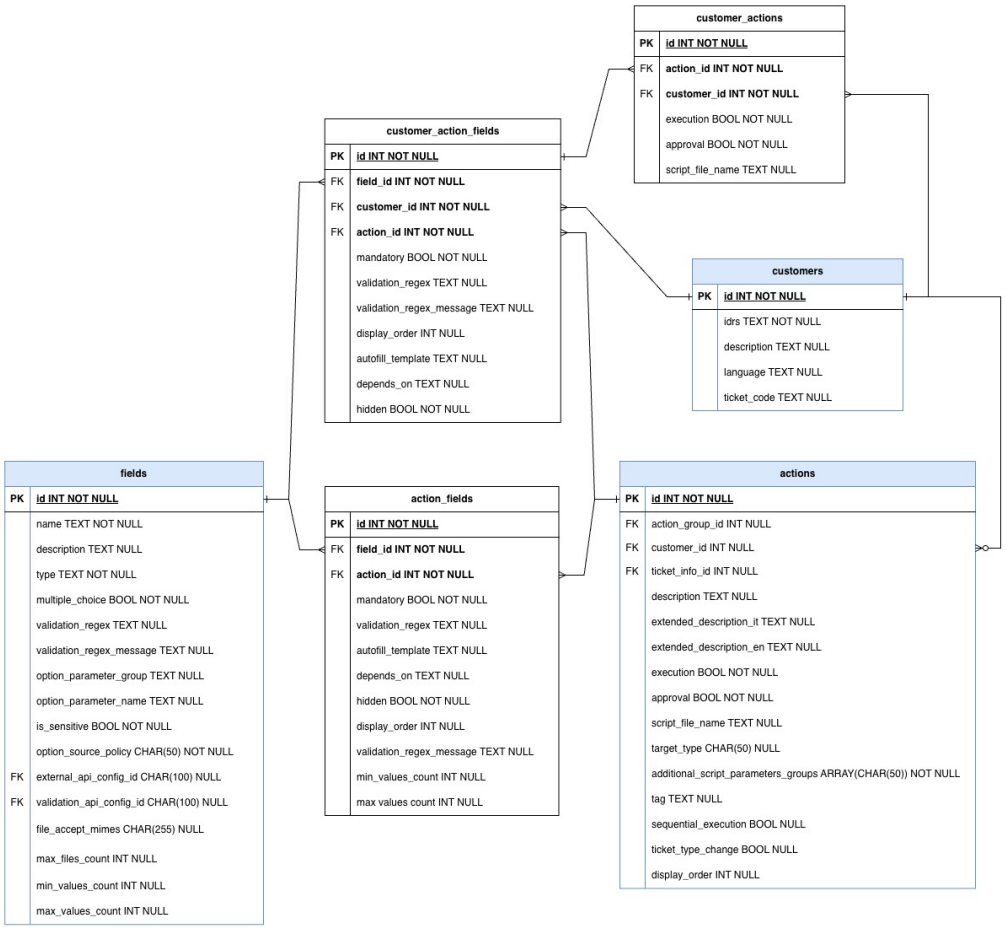

Figure 3.4: ER Diagram of the Three-Level Catalog Configuration Hierarchy

Key Architectural Patterns

Service Block-Based Action Association: Customer access to actions is determined through service blocks derived from contract information rather than granular one-to-one associations, significantly reducing administrative overhead.

Field Type System: The catalog implements 14 distinct field types including simple types (text, password, email), complex types (text lists, select dropdowns, UPN inputs), and specialized validation for each type.

Templating System: Jinja2 templating enables dynamic behavior throughout the configuration layer. Field autofill templates define patterns for prepopulating values based on other fields, as shown in Listing 5.

```
# Template example for UPN autofill
autofill_template = "{{ first_name }}.{{ last_name }}"
# Renders to: "john.doe" when first_name="john", last_name="doe"

# API endpoint pattern with templating
endpoint_pattern = "/api/customers/{{ idrs }}/adusers?domain={{ domain }}"
# ...
```

Listing 5: Jinja2 Templating Examples for Dynamic Field Configuration

3.3.4 Core Execution Workflows

Field Validation

The validation system, centralized in `ServiceRequestValidationMixin`, implements comprehensive pre-submission validation. The architecture separates global validators (operating on complete field sets) from field validators (examining individual field-value pairs). Listing 6 demonstrates the regex validation implementation.

```
def validate_field_value_regex(
    field: Field, value: str | None, **_kwargs
) -> None:
    if field.validation_regex and value:
        if not re.fullmatch(field.validation_regex, value):
            raise serializers.ValidationError({
                "fields_validation": [
                    f"The value '{value}' does not match pattern"
                ]
            })
```

Listing 6: Field Validation Using Regex Patterns

Validation progresses through multiple layers: syntactic validation via regex patterns, configuration-based validation, option validation for select fields, and specialized validation for external endpoints. The framework implements

error aggregation, continuing validation after detecting failures to accumulate all errors before returning consolidated responses.

Asynchronous Execution Flow

The execution workflow implements event-driven task orchestration through Celery. The flow implements a state machine (NEW, WAIT4APPROVAL, APPROVED, PENDING, EXECUTED, SUSPENDED, ERROR, REJECTED) coordinated through six specialized tasks. Listing 7 shows the structure of a typical Celery task.

```python
@shared_task()
def manage_new_service_request_and_ticket(sr_id: int):
    sr = ServiceRequest.objects.get(id=sr_id)

    if sr.action.requires_approval:
        sr.status = "WAIT4APPROVAL"
        sr.save()
    else:
        sr.status = "APPROVED"
        # ...
        execute_service_request.delay(sr_id)
```

Listing 7: Celery Task for Service Request State Management

Script execution occurs through Ansible playbooks implementing a five-phase pattern: preparation, file transfer, execution, logging, and cleanup. This maintains transparency for script developers while providing centralized execution logging.

Master Data Import Pipeline

The master data import subsystem eliminates real-time LDAP query latency through a scheduled export-materialize-query pattern. Listing 8 demonstrates the differential synchronization logic.

3.3.5 Observability and Monitoring

The system implements comprehensive observability through multiple layers: New Relic APM for transaction monitoring, structured logging with automatic New Relic Logs integration, health checks via the `/ht/` endpoint, and Prometheus metrics exposure. The `@monitor_job` decorator wraps scheduled tasks as shown in Listing 9.

3.4 Frontend Application Implementation

The SRM frontend comprises two Vue.js 3 applications: an end-user service request creation interface and an administrative configuration platform.

```python
@transaction.atomic
def process_users_for_customer(customer_id: int, user_data_list):
    logon_names_from_file = set(
        (ud["domain"], ud["logon_name"]) for ud in user_data_list
    )
    existing_users = {
        (user.domain, user.logon_name): user
        for user in AdUser.objects.filter(customer_id=customer_id)
    }

    users_to_create = []
    users_to_update = []
    users_to_delete = []
    # ... differential logic

    if users_to_create:
        AdUser.objects.bulk_create(users_to_create, batch_size=1000)
    if users_to_update:
        AdUser.objects.bulk_update(users_to_update,
            fields=["domain", "distinguished_name"], batch_size=1000)
```

Listing 8: Differential Synchronization for Master Data Import

```python
def monitor_job(func):
    @wraps(func)
    def wrapper(*args, **kwargs):
        application = newrelic.agent.register_application()
        start_time = datetime.now()

        try:
            return_val = func(*args, **kwargs)
            # Send completion event to New Relic
        except Exception as err:
            # Send error event with traceback
            raise
        finally:
            end_time = datetime.now()
            # Log duration and status
    return wrapper
```

Listing 9: Job Monitoring Decorator for Observability

3.4.1 Technology Stack

Both applications leverage Vue 3 Composition API with TypeScript for type safety [43]. State management is implemented through Pinia [22], and the component architecture utilizes a custom design system library (ElmecUIX).

3.4.2 Service Request Creation Interface

The end-user interface implements a dynamic form builder rendering forms based on backend API responses. A single GET request retrieves complete action definitions including resolved field configurations. Listing 10 shows the client-side validation rules generation.

```
const formRules = computed(() => {
  const rules: Record<string, any[]> = {}

  for (const af of action.value.action_fields) {
    const fieldName = af.field.name
    if (!af.mandatory && !af.validation_regex) continue

    rules[fieldName] = []
    if (af.mandatory) {
      rules[fieldName].push({
        required: true,
        message: t("general.required")
      })
    }
    if (af.validation_regex) {
      rules[fieldName].push({
        validator: (rule, value, callback) => {
          const regex = new RegExp(af.validation_regex)
          if (!regex.test(value)) {
            callback(new Error(af.validation_regex_message))
          } else {
            callback()
          }
        }
      })
    }
  }
  return rules
})
```

Listing 10: Dynamic Form Validation Rules in Vue.js

The component architecture separates concerns through parent-child relationships. Field dependency resolution is implemented through reactive watchers monitoring form data changes. External API option caching minimizes redundant backend calls through URL tracking and result storage.

3.4.3 Administrative Configuration Platform

The administrative interface (in design phase) will provide CRUD operations for managing catalog hierarchy components with form-based editors, visual representation of override relationships, and preview functionality enabling administrators to test configurations before deployment.

The frontend architecture achieves dynamic, configuration-driven interfaces that adapt to catalog changes without code modifications, enabling rapid iteration on service request workflows.

Chapter 4

Coexistence and Gradual Migration Strategy

The transition from the legacy SRP system to the new ServiceRequestManager architecture represents a substantial technical and operational challenge that requires careful planning to minimize business disruption while ensuring data integrity and system reliability. Rather than implementing a disruptive "big bang" migration, the project adopts a phased coexistence strategy that enables gradual migration of customers and service request types while maintaining full operational continuity.

4.1 Migration Strategy Overview

The migration architecture implements a dual-flow execution model where both SRP and SRM systems operate simultaneously, with individual service request actions configured to route through either the legacy or modern execution pipeline based on the `target_type` attribute of the Action model, as shown in Listing 11.

```
class ActionTargetTypes(Enum):
    DOMAIN_DC = "From SR Domain (Domain Controller)"
    DOMAIN_EXC = "From SR Domain (Exchange Server)"
    INTERNAL_WORKER = "Internal Worker"
    INTERNAL_API = "API Call"
    SRP = "SRP Agent (Domain Controller)"  # Legacy flow
```

Listing 11: Action Target Type Enumeration for Dual-Flow Routing

Actions configured with `target_type = SRP` continue routing through the legacy SRP Agent execution mechanism, preserving the existing workflow without modification. Actions configured with any other target type utilize the new Ansible-based execution pipeline through Provision Prime, benefiting from improved logging, error handling, and state management capabilities introduced in SRM.

This per-action migration control provides several strategic advantages: it enables pilot deployments on specific customers or action types; it maintains business continuity by ensuring that any discovered issues affect only the subset of actions migrated to the new flow; it supports progressive learning;

and it provides fallback capability, as actions can be reverted to SRP execution if critical issues are discovered.

The primary architectural trade-off is that the catalog configuration must be maintained in the SRP database during the coexistence period, as SRP lacks the sophisticated configuration features introduced in SRM (three-level hierarchy, external API configurations, template-based autofill). The SRM system operates as a read-only consumer of catalog data from SRP, applying its advanced configuration features as an overlay on the imported base catalog.

4.2 Catalog Synchronization from SRP to SRM

The catalog synchronization mechanism implements a unidirectional data flow where SRP serves as the authoritative source of catalog definitions, with SRM periodically importing and enriching this data. This synchronization is executed through the `sync_srp_catalog` Django management command, scheduled as a Kubernetes CronJob for daily execution at 01:00 AM, as shown in Listing 12.

```
@monitor_job
def handle(self, *args, **kwargs):
    with disable_auditlog(), pyodbc.connect(SRP_CONN_STR) as conn:
        cursor = conn.cursor()
        sync_action_types(cursor)
        sync_action_categories(cursor)
        sync_action_groups(cursor)
        sync_customers(cursor)
        sync_actions(cursor)
        sync_config_parameters(cursor)
        sync_fields(cursor)
        # ...
        reset_sequences(app_label="catalog")
```

Listing 12: Catalog Synchronization Job Structure

4.2.1 Synchronization Architecture

The synchronization process establishes a direct ODBC connection to the SRP SQL Server database and executes a series of import operations that reconstruct the complete catalog hierarchy in the SRM PostgreSQL database. The process imports entities in a carefully ordered sequence that respects foreign key dependencies: action types → categories → groups → customers → actions → configuration parameters → fields → customer-specific customizations.

The synchronization executes with audit logging disabled to prevent creation

of thousands of audit log entries for bulk import operations, improving performance and reducing database bloat. Each synchronization function implements a consistent pattern: query the source table in SRP, construct Django model instances for new or modified records, and execute bulk create/update operations.

4.2.2 Differential Synchronization Logic

The synchronization functions implement sophisticated differential logic that minimizes database operations by detecting actual changes rather than blindly recreating all data. Listing 13 demonstrates the pattern used across synchronization functions.

```
records = [dict(zip(columns, row)) for row in cursor.fetchall()]
source_ids = {int(r["IDCategory"]) for r in records}

existing_categories = {
    ac.id: ac for ac in ActionCategory.objects.filter(
        id__in=source_ids
    )
}

to_create = []
to_update = []

for record in records:
    category_id = int(record["IDCategory"])
    if category_id in existing_categories:
        obj = existing_categories[category_id]
        obj.description = record["DescCategory"]
        # ...
        to_update.append(obj)
    else:
        to_create.append(ActionCategory(
            id=category_id,
            description=record["DescCategory"]
            # ...
        ))

if to_create:
    ActionCategory.objects.bulk_create(to_create)
if to_update:
    ActionCategory.objects.bulk_update(to_update,
        ['description', 'display_order'])
```

Listing 13: Differential Synchronization Pattern with Bulk Operations

This approach constructs in-memory dictionaries of existing records keyed by primary key, then iterates through source records to segregate them into create and update lists. Bulk operations execute with batch sizes appropriate for PostgreSQL, significantly outperforming individual save operations. An initial full synchronization strategy that deleted and recreated all records at

each execution proved inefficient with SRP's large data volumes, resulting in execution times around 2 hours. The current differential logic significantly reduces database load, bringing average execution times down to approximately 20 seconds.

4.2.3 Post-Import Enrichment

Following the base catalog import from SRP, the synchronization process executes several enrichment operations that apply SRM-specific configurations: API configuration synthesis creates `ExternalAPIConfiguration` records defining option sources for select fields; field duplication resolution identifies and resolves scenarios where multiple fields with identical names exist within the same action scope; HDA field configuration synthesizes metadata for external fields and classification fields; and customer-specific customizations translate SRP configurations into SRM's three-level hierarchy.

The synchronization concludes by resetting PostgreSQL auto-increment sequences to prevent primary key conflicts on subsequent manual record creation.

4.3 Execution History Synchronization

While catalog synchronization flows unidirectionally from SRP to SRM, execution history must be merged from both systems to provide unified visibility. The `sync_srp_sr_history` command implements this bidirectional synchronization, scheduled to execute at 02:00 AM (one hour after catalog sync).

The execution history synchronization imports three primary entity types: service request headers, field values, and execution logs. The synchronization is filtered by a configurable date threshold to avoid importing the complete historical archive, defaulting to the first day of the current month, as shown in Listing 14.

```
start_date = os.getenv(
    "SRP_SR_SYNC_START_DATE",
    datetime.date.today().replace(day=1).strftime("%Y-%m-%d")
)
cursor.execute(
    f"SELECT * FROM TBSR_Headers WHERE Date > '{start_date}'"
)
# ... process service requests
sync_service_requests(cursor)
sync_service_request_fields(cursor)
sync_service_request_logs(cursor)
```

Listing 14: Execution History Synchronization with Date Filtering

Service request field values imported from SRP must respect SRM's enhanced

security model where sensitive fields are encrypted at rest. The synchronization implements transparent encryption during import, checking each field's sensitivity flag and applying encryption to values from sensitive fields. Execution logs stored in SRP use a numeric type code system incompatible with SRM's enumerated log types, requiring a mapping table that translates SRP codes to SRM equivalents (Creation, Approval, Execution Started, Execution Completed, etc.).

4.4 Dual Execution Flow Management

The coexistence architecture's core mechanism is the dynamic routing of service request execution based on action configuration. When a service request is created through the SRM API, the system evaluates the action's `target_type` to determine whether to invoke the new or legacy execution pipeline, as illustrated in Listing 15.

```
is_srp_flow = action.target_type == ActionTargetTypes.SRP.name

# Create HDA ticket (common to both flows)
response_ticket = mysupport.create_ticket(
    codicli=codicli, idrs=customer.idrs, body=body
)
ticket_id = response_ticket.get("ID")

if is_srp_flow:
    # Legacy flow: SRP will create the SR entity
    logger.info(f"SRP flow detected. Ticket {ticket_id} created.")
    return ServiceRequest(
        action=action, customer=customer,
        ticket_reference=ticket_id
    )
else:
    # Modern flow: SRM creates SR and triggers Celery tasks
    validated_data["ticket_reference"] = ticket_id
    service_request = super().create(validated_data)
    # ... trigger async execution
    return service_request
```

Listing 15: Dual Execution Flow Routing Logic

For actions configured with `target_type = SRP`, the serializer creates the HDA ticket but returns a non-persisted ServiceRequest object. The actual service request record will be created by SRP's legacy REST API when HDA invokes it, then synchronized into SRM's database by the nightly job. For actions with any non-SRP target type, the ServiceRequest record is persisted immediately with NEW status, and the asynchronous Celery task chain is initiated.

The dual-flow architecture remains transparent to frontend applications, which interact exclusively with SRM APIs regardless of the underlying execution

mechanism, ensuring that frontend applications require no modifications as actions are gradually migrated.

4.5 Pilot Strategy and Progressive Rollout

The migration strategy implements a risk-managed progression that validates the new architecture incrementally before full deployment, organized into four distinct phases.

4.5.1 Phase 1: Internal Pilot

The initial migration phase configures a small subset of actions used exclusively by internal IT operations teams to execute through the SRM pipeline. These actions are selected based on low execution volume, non-critical business processes, and comprehensive logging requirements. Internal teams provide direct feedback on execution behavior, enabling rapid iteration on issues discovered during real-world usage.

4.5.2 Phase 2: Customer Pilot

Following successful internal validation, the migration expands to a carefully selected cohort of pilot customers chosen based on technical sophistication, diverse action usage patterns, and strong relationship with the company. Each pilot customer has a subset of their actions migrated to SRM execution, typically starting with low-risk operations before progressing to more critical provisioning workflows.

The pilot phase implements enhanced monitoring including dedicated Grafana dashboards, daily review meetings, and direct communication channels with pilot customers. Issues discovered during pilot are addressed through hotfix deployments, with actions reverted to SRP execution if critical problems cannot be resolved quickly.

4.5.3 Phase 3: Progressive Expansion

Successful pilot completion triggers progressive expansion where additional customer cohorts are migrated on a weekly cadence. Each expansion wave includes a defined set of customers and actions, with execution metrics monitored for anomalies before proceeding to the next wave. The expansion velocity is dynamically adjusted based on observed success rates: high success rates (>99%) enable acceleration, while elevated error rates trigger pause for investigation.

The rollout maintains a progressive migration of action types from simple to complex: read-only information queries migrate first, followed by non-

destructive provisioning operations, then critical operations, and finally complex multi-step workflows.

4.5.4 Phase 4: SRP Decommissioning

Migration completion occurs when all customers and actions have successfully transitioned to SRM execution, validated through sustained periods (30+ days) of stable operation without SRP fallback. The SRP system transitions to read-only mode, the catalog synchronization job is deactivated, and finally the SRP infrastructure is decommissioned after a retention period ensuring all audit and compliance requirements are satisfied.

4.6 Data Integrity and Consistency Mechanisms

The coexistence architecture implements several mechanisms to maintain data integrity and consistency during the migration period.

4.6.1 Transactional Boundaries and Idempotency

Synchronization operations execute within database transaction contexts ensuring atomicity: either all records in a synchronization batch commit successfully, or none do. The synchronization logic is designed for idempotency, enabling safe re-execution in case of failures. Each sync function can be invoked multiple times with identical results, as record identification is based on deterministic primary keys from SRP.

4.6.2 Referential Integrity and Audit Trail

Foreign key relationships are validated before record creation to prevent orphaned references. The synchronization order ensures parent entities exist before children, while optional relationships are nulled when references cannot be resolved. While audit logging is disabled during bulk synchronization for performance, the synchronization jobs themselves are comprehensively logged through the monitoring infrastructure, with execution metrics recorded in New Relic.

4.7 Operational Considerations and Migration Benefits

The coexistence strategy introduces several operational challenges that require careful management during the migration period.

4.7.1 Operational Challenges

The primary challenges include: dual maintenance burden requiring catalog changes in SRP that propagate through synchronization; synchronization latency introducing up to 24-hour delays between SRP changes and SRM visibility; execution history fragmentation requiring staff training to query SRM exclusively; and performance considerations necessitating scheduled execution during low-traffic periods.

4.7.2 Risk Mitigation and Benefits

The coexistence and gradual migration strategy provides a pragmatic path to modernization while maintaining operational continuity. The dual-flow architecture enables incremental validation of the new system in production environments, building confidence through measured progression rather than high-risk wholesale replacement. The phased rollout provides operations teams with extended periods to develop familiarity with the new system architecture, monitoring tools, and troubleshooting procedures.

The coexistence strategy eliminates migration-related service disruptions by maintaining the legacy system as a fully functional fallback throughout the transition period. This approach ensures that service level agreements are maintained throughout the migration, avoiding the business impact of failed "big bang" migrations. The gradual migration provides real-world validation of architectural decisions under production loads, enabling refinement based on operational feedback before full deployment.

While the strategy introduces operational complexity through the need for synchronization and dual-system maintenance, these costs are justified by the substantial risk reduction and business continuity assurance provided by the gradual approach. The coexistence period represents a necessary investment in migration safety that protects both the company and its customers from the substantial risks inherent in large-scale system replacements.

Chapter 5

Recommendation system through LLM and Retrieval Augmented Generation

5.1 Motivation and Problem Statement

5.1.1 Limitations of Traditional Catalog Navigation

5.1.2 Natural Language as an Interface for Service Requests

5.1.3 Explainability and Trust in Enterprise Environments

5.2 Large Language Models in Recommendation Systems

5.2.1 Overview of Large Language Models

5.2.2 LLMs for Intent Understanding and Action Selection

5.2.3 Limitations of Naked LLMs

5.3 Retrieval Augmented Generation (RAG)

5.3.1 Core Concepts of Retrieval Augmented Generation

5.3.2 How RAG Works

5.3.3 Problems Solved by RAG

5.4 Design of the RAG-based Recommendation System in SRM

5.4.1 High-Level Architecture

5.4.2 Embedding Strategy

Conclusions

- Reflections on the internship experience and acquired skills
- Considerations on the future of the project and potential developments

Bibliography

[1] *Active Directory Organizational Unit (OU)*. URL: https://blog.netwrix.com/2024/02/19/active-directory-organizational-unit-ou/ (cit. on p. 23).

[2] Mashel Albarak and Rami Bahsoon. *Prioritizing Technical Debt in Database Normalization Using Portfolio Theory and Data Quality Metrics*. 2018. arXiv: 1801.06989 [cs.SE]. URL: https://arxiv.org/abs/1801.06989 (cit. on p. 16).

[3] Charles Anderson. "Docker [Software engineering]". In: *IEEE Software* 32.3 (2015), pp. 102–c3. DOI: 10.1109/MS.2015.62 (cit. on p. 28).

[4] *Ansible Documentation*. URL: https://docs.ansible.com/ansible/latest/dev_guide/overview_architecture.html (cit. on pp. 33, 34, 43).

[5] *Argo CD - Declarative GitOps CD for Kubernetes*. URL: https://argo-cd.readthedocs.io/en/stable/ (cit. on p. 40).

[6] Michael R. Blaha. "Referential Integrity Is Important For Databases". In: 2005. URL: https://api.semanticscholar.org/CorpusID:6831872 (cit. on p. 16).

[7] *Celery - Distributed Task Queue*. URL: https://docs.celeryq.dev/en/stable/getting-started/introduction.html (cit. on p. 43).

[8] *Connection Pooling for Postgres using PG Bouncer*. URL: https://medium.com/@pablo.lopez.santori/connection-pooling-for-postgres-using-pg-bouncer-175bc1607db2 (cit. on p. 43).

[9] *Cryptsetup and LUKS - open-source disk encryption*. URL: https://gitlab.com/cryptsetup/cryptsetup (cit. on p. 11).

[10] Lorenzo De Lauretis. "From Monolithic Architecture to Microservices Architecture". In: *2019 IEEE International Symposium on Software Reliability Engineering Workshops (ISSREW)*. 2019, pp. 93–96. DOI: 10.1109/ISSREW.2019.00050 (cit. on p. 24).

[11] *Django - The web framework for perfectionists with deadlines*. URL: https://www.djangoproject.com/ (cit. on p. 42).

[12] *Django REST framework*. URL: https://www.django-rest-framework.org/ (cit. on p. 42).

[13] *DMI Infrastructure*. Internal repository, not publicly accessible. URL: https://git.elmec.com/publicautomation/dmi-infrastructure.git (cit. on pp. 8, 10, 11).

[14] Roy T. Fielding. *Architectural Styles and the Design of Network-Based Software Architectures*. University of California, Irvine, 2000 (cit. on p. 26).

[15] Paul J. Deitel Harvey M. Deitel. *C++. Fondamenti di programmazione*. Apogeo, 2005 (cit. on p. 16).
[16] *HDA - Help Desk Advanced*. URL: https://pat.eu/en/products/helpdeskadvanced/ (cit. on p. 27).
[17] *K3s - Lightweight Kubernetes*. URL: https://docs.k3s.io/ (cit. on p. 11).
[18] *Kubernetes: Production-Grade Container Orchestration*. URL: https://kubernetes.io/docs/concepts/overview/ (cit. on pp. 27, 29).
[19] *Memcached - A distributed memory object caching system*. URL: https://docs.memcached.org/ (cit. on p. 43).
[20] Severi Peltonen, Luca Mezzalira, and Davide Taibi. *Motivations, Benefits, and Issues for Adopting Micro-Frontends: A Multivocal Literature Review*. 2021. arXiv: 2007.00293 [cs.SE]. URL: https://arxiv.org/abs/2007.00293 (cit. on pp. 12, 24).
[21] *PEP 8 – Style Guide for Python Code*. URL: https://peps.python.org/pep-0008/ (cit. on p. 40).
[22] *Pinia - The intuitive store for Vue.js*. URL: https://pinia.vuejs.org/introduction.html (cit. on p. 49).
[23] *Pod Quality of Service Classes*. URL: https://kubernetes.io/docs/concepts/workloads/pods/pod-qos/ (cit. on p. 41).
[24] *Provision Prime*. Internal repository, not publicly accessible. URL: https://git.elmec.com/innovation/development/provisionprime.git (cit. on pp. 23, 24, 27).
[25] *Refactoring Guru - Strategy Pattern*. URL: https://refactoring.guru/design-patterns/strategy (cit. on p. 34).
[26] David K. Rensin. *Kubernetes - Scheduling the Future at Cloud Scale*. 2015, All. URL: http://www.oreilly.com/webops-perf/free/kubernetes.csp (cit. on pp. 29, 30).
[27] Elmec Informatica S.p.A. *Azioni Gestione SRP 4.0*. Internal document, not publicly available., 2016 (cit. on pp. 3, 5, 6, 26).
[28] Elmec Informatica S.p.A. *Requirements definition meeting for the new system*. Internal meeting. Held with the stakeholders and developers of the old system to gather requirements for the new system. Apr. 2025 (cit. on pp. 8, 16, 19–24).
[29] Elmec Informatica S.p.A. *Service Request Portal*. Internal document, not publicly available., 2016 (cit. on pp. 8, 9, 22).
[30] *Semgrep App Security Platform | AI-Assisted SAST, SCA and Secrets Detection*. URL: https://semgrep.dev/ (cit. on p. 40).
[31] *SQL Server Integration Services*. URL: https://learn.microsoft.com/en-us/sql/integration-services/sql-server-integration-services (cit. on p. 14).
[32] *SRP Internal Worker Windows Service*. Internal repository, not publicly accessible. URL: https://git.elmec.com/automationcloud/srpinternalworker_windowsservice.git (cit. on p. 10).

[33] *SRP Web Service*. Internal repository, not publicly accessible. URL: https://git.elmec.com/automationcloud/SRP_WebService.git (cit. on p. 9).

[34] *SRP Web Service REST*. Internal repository, not publicly accessible. URL: https://git.elmec.com/automationcloud/srp_webservicerest.git (cit. on pp. 9, 19, 22, 24).

[35] *SRP Windows Service*. Internal repository, not publicly accessible. URL: https://git.elmec.com/automationcloud/SRP_WindowsService.git (cit. on pp. 7, 10, 16, 22–24).

[36] Michael Stonebraker, Lawrence Rowe, and Michael Hirohama. "The Implementation Of Postgres". In: *Knowledge and Data Engineering, IEEE Transactions on* 2 (Apr. 1990), pp. 125–142. DOI: 10.1109/69.50912 (cit. on p. 35).

[37] *Stored Procedures (Database Engine)*. URL: https://learn.microsoft.com/en-us/sql/relational-databases/stored-procedures/stored-procedures-database-engine (cit. on p. 14).

[38] Jan Tretmans and Louis Verhaard. "A Queue Model Relating Synchronous and Asynchronous Communication". In: *Protocol Specification, Testing and Verification, XII*. Ed. by R.J. LINN and M.Ü. UYAR. IFIP Transactions C: Communication Systems. Amsterdam: Elsevier, 1992, pp. 131–145. ISBN: 978-0-444-89874-6. DOI: https://doi.org/10.1016/B978-0-444-89874-6.50015-5. URL: https://www.sciencedirect.com/science/article/pii/B9780444898746500155 (cit. on p. 27).

[39] *Trivy - The all-in-one open source security scanner*. URL: https://trivy.dev/ (cit. on p. 40).

[40] *TypeScript: JavaScript with Syntax for Types*. URL: https://www.typescriptlang.org/docs/handbook/typescript-from-scratch.html (cit. on p. 30).

[41] Brecht Van de Vyvere, Pieter Colpaert, and Ruben Verborgh. "Comparing a Polling and Push-Based Approach for Live Open Data Interfaces". In: *Web Engineering*. Ed. by Maria Bielikova, Tommi Mikkonen, and Cesare Pautasso. Cham: Springer International Publishing, 2020, pp. 87–101. ISBN: 978-3-030-50578-3 (cit. on p. 22).

[42] *Vault - Manage Secrets and Protect Sensitive Data*. URL: https://developer.hashicorp.com/vault (cit. on p. 41).

[43] *Vue.js v3 Documentation*. URL: https://vuejs.org/guide/introduction (cit. on pp. 30, 31, 49).

[44] *What is a Container?* URL: https://www.docker.com/resources/what-container/ (cit. on p. 29).

[45] *What is referential integrity and what does it look like in practice?* URL: https://www.y42.com/blog/referential-integrity (cit. on p. 16).

[46] *Xenon - Monitoring tool based on radon*. URL: https://github.com/rubik/xenon (cit. on p. 40).